

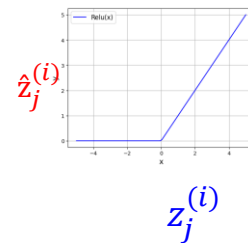
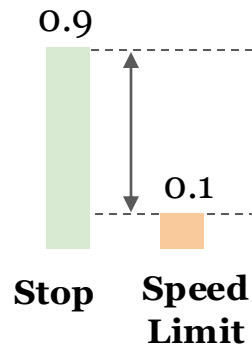
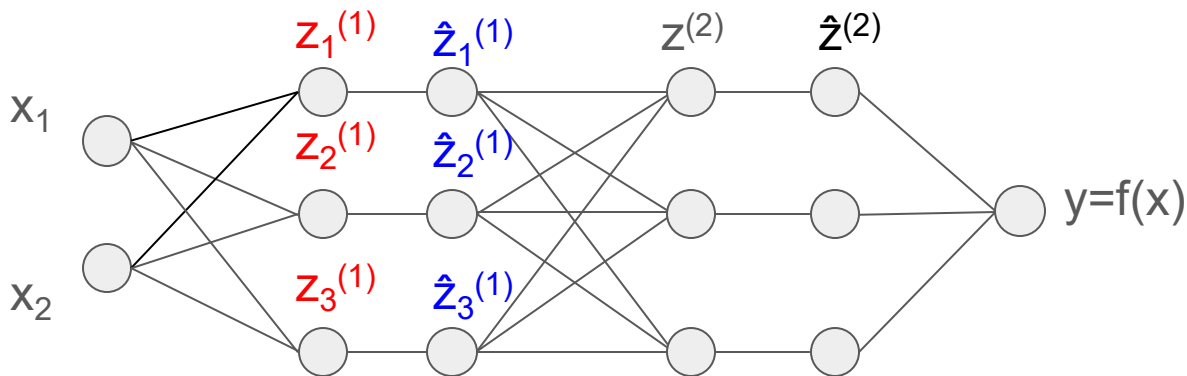
Neural Network Verification

Integer and Linear Programming Formulations

Prof. Sayan Mitra

mitras@Illinois.edu

Review: Neural network verification as a satisfiability problem



To verify for all $x \in S$, $y > 0 \wedge y = f(x)$ we solve
 Does there exist x , s.t. $x \in S \wedge y \leq 0 \wedge y = f(x)$

Input domain under consideration

Negation of the desired property

Defines the neural network

Verification of neural networks

Satisfiability problem: $\exists x \in S \wedge y \leq 0 \wedge y = f(x)$

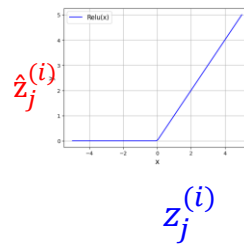
$x_i \leq u_i \wedge x_i \geq l_i$ for each dimension of x

$((z_j^{(i)} \geq 0 \wedge \hat{z}_j^{(i)} = z_j^{(i)}) \vee (z_j^{(i)} < 0 \wedge \hat{z}_j^{(i)} = 0))$ for each ReLU neuron

$z_1 = W^{(1)} x \wedge z^{(2)} = W^{(2)} \hat{z}^{(1)} \wedge y = w^{(3)T} \hat{z}^{(2)} \wedge y \leq 0$

Add all clauses to the formula and solve using DPLL(T) with Linear Real Arithmetic.

In general this is very slow!



Reluplex: Decision Procedure for ReLU NNs

Input: F in Reluplex form

Output: $\exists \mathbf{x} \in \mathbb{R}^m$ such that $\mathbf{x} \models F$?

- Delay case splitting on ReLUs
- In the worst case it is still exponential, but has been shown to be empirically better than DPLL^{LRA}

Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks

[Guy Katz](#), [Clark Barrett](#), [David Dill](#), [Kyle Julian](#), [Mykel Kochenderfer](#), 2017

Reluplex Decision Procedure for ReLU NNs

Input: F in Reluplex form

Output: $\exists \mathbf{x} \in \mathbb{R}^m$ such that $\mathbf{x} \models F$?

Reluplex form (Recall Simplex form)

- Equations (same as Simplex)
- Bounds (same as Simplex)
- Relu: $x_i = \text{relu}(x_j)$

Given conjunction of inequalities and ReLU constraints we can convert them to Reluplex form and add $x_i \geq 0$.

Reluplex

Input: A formula F in Reluplex form

Output: $x \models F$ or UNSAT

$x := \langle x_i \mapsto 0 \rangle$; $F' := \text{non ReLU part of } F$

while true do

$r := \text{Simplex}(F', x)$

 if r is unsat then return UNSAT

 else if $r \models F$ then return r

// Handle violated ReLU constraints

 Let $x_i = \text{relu}(x_j) \in F$ such that $x[x_i] \neq \text{relu}(x[x_j])$

// Case split

 If $u_j > 0, l_j < 0$ and $x_i = \text{relu}(x_j)$ considered $> \tau$ times

Reluplex

Input: A formula F in Reluplex form

Output: $x \models F$ or UNSAT

$x := \langle x_i \mapsto 0 \rangle$; $F' := \text{non ReLU part of } F$

while true do

$r := \text{Simplex}(F', x)$

 if r is unsat then return UNSAT

 else if $r \models F$ then return r

// Handle violated ReLU constraints

 Let $x_i = \text{relu}(x_j) \in F$ such that $x.x_i \neq \text{relu}(x.x_j)$

 if x_i is basic then pivot x_i with non-basic variable x_k where $k \neq j$ and $c_{ik} \neq 0$

 if x_j is basic then pivot x_j with non-basic variable x_k where $k \neq i$ and $c_{jk} \neq 0$

$x.x_i := \text{relu}(x.x_j)$ OR $x.x_j := x.x_i$

// Case split

 If $u_j > 0, l_j < 0$ and $x_i = \text{relu}(x_j)$ considered $> \tau$ times

$r_1 = \text{Reluplex}(F \wedge x_j \geq 0 \wedge x_j = x_j)$

$r_2 = \text{Reluplex}(F \wedge x_j \leq 0 \wedge x_i = 0)$

 if $r_1 = r_2 = \text{unsat}$ then return UNSAT

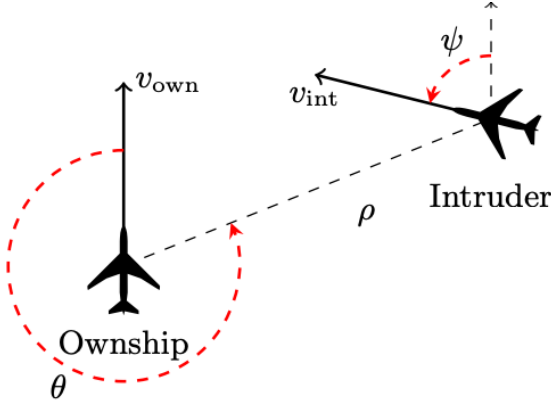
 else if $r_1 = \text{sat}$ then return r_1 else return r_2

Termination

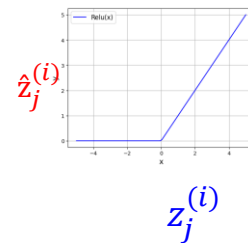
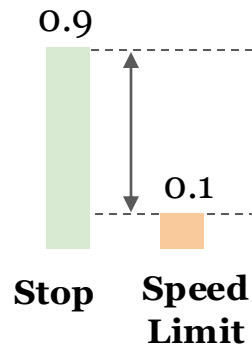
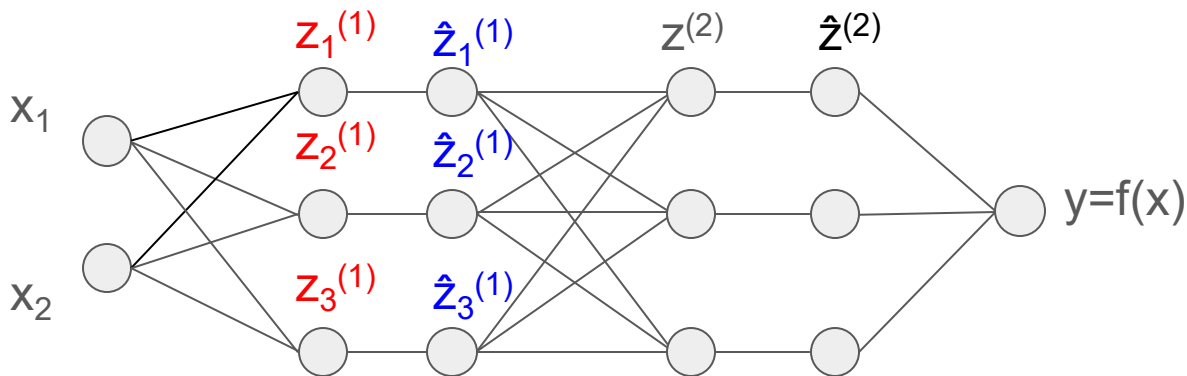
- The last part of the algorithm ensures termination
- Otherwise, the algorithm could loop forever between fixing relu constraints and Simplex
- $F \equiv x_i = \text{relu}(x_j)$ is split into two cases
 - $F_1 \equiv x_j \geq 0 \wedge x_i = x_j$
 - $F_2 \equiv x_j \leq 0 \wedge x_i = 0$
 - $F' \equiv (F \wedge F_1) \vee (F \wedge F_2)$

Performance of Reluplex on ACAS

	φ_1	φ_2	φ_3	φ_4	φ_5	φ_6	φ_7	φ_8
CVC4	-	-	-	-	-	-	-	-
Z3	-	-	-	-	-	-	-	-
Yices	1	37	-	-	-	-	-	-
MathSat	2040	9780	-	-	-	-	-	-
Gurobi	1	1	1	-	-	-	-	-
Reluplex	8	2	7	7	93	4	7	9



Review: Neural network verification as a satisfiability problem



Does there exist x , s.t. $x \in S \wedge y \leq 0 \wedge y = f(x)$

Input domain under consideration

Negation of the desired property

Defines the neural network

Verification of neural networks

Satisfiability problem: $\exists x \in S \wedge y \leq 0 \wedge y = f(x)$

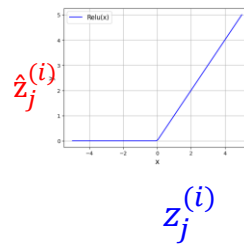
$x_i \leq u_i \wedge x_i \geq l_i$ for each dimension of x

$((z_j^{(i)} \geq 0 \wedge \hat{z}_j^{(i)} = z_j^{(i)}) \vee (z_j^{(i)} < 0 \wedge \hat{z}_j^{(i)} = 0))$ for each ReLU neuron

$z_1 = W^{(1)} x \wedge z^{(2)} = W^{(2)} \hat{z}^{(1)} \wedge y = w^{(3)T} \hat{z}^{(2)} \wedge y \leq 0$

Add all clauses to the formula and solve using DPLL(T) with Linear Real Arithmetic.

In general this is very slow!



How does a SMT solver solve this problem?

Obtain the abstract version of the problem

$x_i \leq u_i \wedge x_i \geq l_i$ (assuming box constraints)

$((z_j^{(i)} \geq 0 \wedge \hat{z}_j^{(i)} = z_j^{(i)}) \vee (z_j^{(i)} < 0 \wedge \hat{z}_j^{(i)} = 0))$ for each ReLU neuron

$z_1 = W^{(1)} x \quad \wedge \quad z^{(2)} = W^{(2)} \hat{z}^{(1)} \quad \wedge \quad y = w^{(3)T} \hat{z}^{(2)} \quad \wedge \quad y \leq 0$

For each ReLU neuron: $((p_j^{(i)} \wedge q_j^{(i)}) \vee (\neg p_j^{(i)} \wedge r_j^{(i)}))$

All other clauses contain only a single literal, and must be set to True

How does a SMT solver solve this problem?

Now convert to CNF form

For each ReLU neuron: $((p_j^{(i)} \wedge q_j^{(i)}) \vee (\neg p_j^{(i)} \wedge r_j^{(i)}))$

Distribution: $((p_j^{(i)} \vee \neg p_j^{(i)}) \wedge (p_j^{(i)} \vee r_j^{(i)}) \wedge (q_j^{(i)} \vee \neg p_j^{(i)}) \wedge (q_j^{(i)} \vee r_j^{(i)}))$

Rewrite: $(p_j^{(i)} \vee r_j^{(i)}) \wedge (\neg p_j^{(i)} \vee q_j^{(i)}) \wedge (q_j^{(i)} \vee r_j^{(i)})$

Observe that when $p_j^{(i)} = \text{True}$, $q_j^{(i)}$ must be true; $p_j^{(i)} = \text{False}$, $r_j^{(i)}$ must be true;

So, the clause $q_j^{(i)} \vee r_j^{(i)}$ is redundant

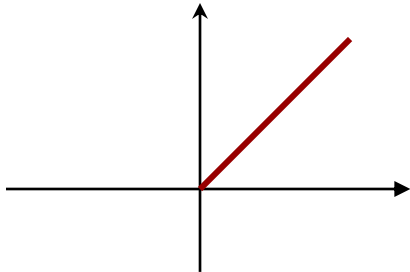
How does a SMT solver solve this problem?

$$(p_j^{(i)} \vee r_j^{(i)}) \wedge (\neg p_j^{(i)} \vee q_j^{(i)})$$

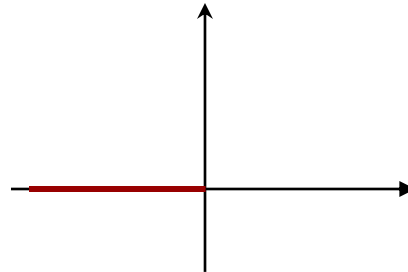
When $p_j^{(i)} = \text{True}$, $q_j^{(i)}$ must be true; $p_j^{(i)} = \text{False}$, $r_j^{(i)}$ must be true;

SAT solver must try both cases of $p_j^{(i)}$

$$p_j^{(i)} = \text{True} \quad (z_j^{(i)} \geq 0)$$



$$p_j^{(i)} = \text{False} \quad (z_j^{(i)} < 0)$$



Why using a SMT solver is very slow?

$$(p_j^{(i)} \vee r_j^{(i)}) \wedge (\neg p_j^{(i)} \vee q_j^{(i)})$$

When $p_j^{(i)} = \text{True}$, $q_j^{(i)}$ must be true; $p_j^{(i)} = \text{False}$, $r_j^{(i)}$ must be true;

SAT solver must make decisions on $p_j^{(i)}$. In a satisfiable solution from DPLL, each $p_j^{(i)}$ is set to True or False, then a theory solver (Simplex) invoked.

There are exponential number of cases here... and modern neural networks can have millions of neurons!

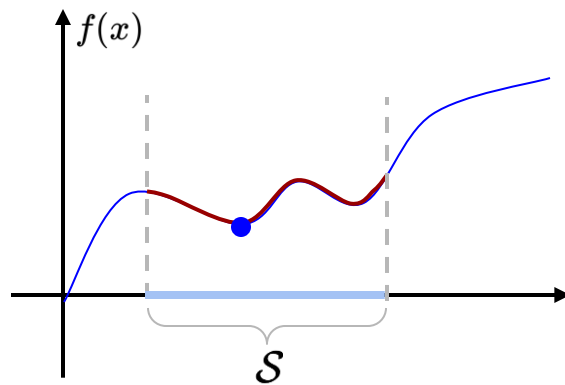
Can we solve the problem without setting every $p_j^{(i)}$?

We will tack this problem from an optimization point of view

Mathematical optimization problems

Given an **objective function** $f: S \rightarrow \mathbb{R}$

Seek an **optimal solution** x^* such that $f(x^*) \leq f(x)$ for all $x \in S$



Some optimization problems are hard, some are easy

Given an **objective function** $f: S \rightarrow \mathbb{R}$

Hardness depends on the properties of f and S . For tractable solving they cannot be arbitrary!

- Easy ones: convex optimization, linear programming, semidefinite programming
 - E.g., in linear programming, objective function and constraints must be linear functions
- Hard ones: integer programming, general quadratic programming, general nonlinear programming, ...

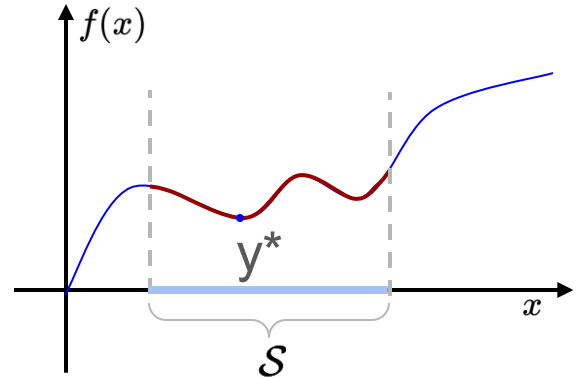
Verification Problem as an optimization problem

$$\exists x \in S \wedge y \leq 0 \wedge y = f(x)$$

Can be solved with the following minimization problem:

$$y^* = \min_{x \in S} f(x)$$

If the optimal objective $y^* \leq 0$, then the original Problem is satisfiable



Verification Problem as an optimization problem

Now we rewrite the neural network verification problem as a **constrained optimization problem** (still using the simple network example):

$$\begin{aligned} & \min y \\ \text{s.t. } & y = w^{(3)T} \hat{z}^{(2)} && \text{(linear layers)} \\ & \hat{z}^{(2)} = \max(z^{(2)}, 0) && \text{(ReLU activation)} \\ & z^{(2)} = W^{(2)} \hat{z}^{(1)} \\ & \hat{z}^{(1)} = \max(z^{(1)}, 0) \\ & z^{(1)} = W^{(1)} x \\ & x_i \leq u_i && \text{(element-wise input bounds)} \\ & x_i \geq li \end{aligned}$$

Verification Problem as an optimization problem

Now we rewrite the neural network verification problem as a **constrained optimization problem** (still using the simple network example):

$$\begin{aligned} & \min y \\ \text{s.t. } & y = w^{(3)T} \hat{z}^{(2)} && \text{(linear constraints)} \\ & \hat{z}^{(2)} = \max(z^{(2)}, 0) && \text{(\textcolor{red}{nonlinear} constraints)} \\ & z^{(2)} = W^{(2)} \hat{z}^{(1)} \\ & \hat{z}^{(1)} = \max(z^{(1)}, 0) \\ & z^{(1)} = W^{(1)} x \\ & x_i \leq u_i && \text{(inputs bounds are also linear constraints)} \\ & x_i \geq l_i \end{aligned}$$

Optimization formulation for ReLU neurons

Let's look at this constraint more carefully: $\hat{z}_j^{(i)} = \max(z_j^{(i)}, 0)$ (for all i, j)

One approach: **mixed integer linear programming (MILP)** formulation

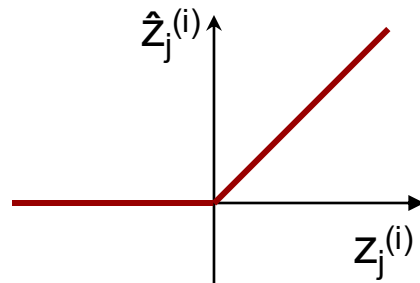
Create an **integer variable** $p_j^{(i)} \in \{0, 1\}$ and rewrite using “big” M number:

$$\hat{z}_j^{(i)} \leq z_j^{(i)} - M_1 (1 - p_j^{(i)})$$

$$\hat{z}_j^{(i)} \leq M_2 p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$



Optimization formulation for ReLU neurons

Let's look at this constraint more carefully: $\hat{z}_j^{(i)} = \max(z_j^{(i)}, 0)$ (for all i, j)

$p_j^{(i)} = 1$ ReLU is active $\hat{z}_j^{(i)} = z_j^{(i)} ; \hat{z}_j^{(i)} \geq 0 ; \hat{z}_j^{(i)} \leq M_2$; for $M_2 \gg 0$ this is no constraint

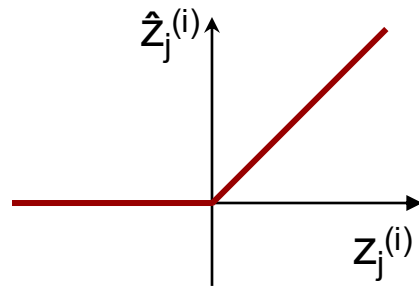
$p_j^{(i)} = 0$ ReLU is inactive $\hat{z}_j^{(i)} = 0 ; \hat{z}_j^{(i)} \geq z_j^{(i)} ; \hat{z}_j^{(i)} \leq z_j^{(i)} - M_1$; for $M_1 \ll 0$ no constraint

$$\hat{z}_j^{(i)} \leq z_j^{(i)} - M_1 (1 - p_j^{(i)})$$

$$\hat{z}_j^{(i)} \leq M_2 p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$



M_1 needs to be small (negative) enough

M_2 needs to be large enough

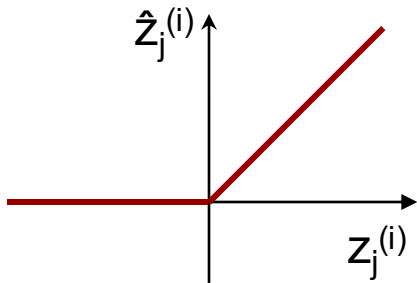
Optimization formulation for ReLU neurons

$$\hat{z}_j^{(i)} \leq z_j^{(i)} - M_1 (1 - p_j^{(i)})$$

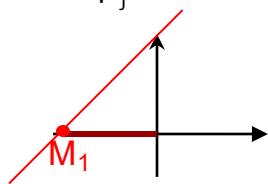
$$\hat{z}_j^{(i)} \leq M_2 p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

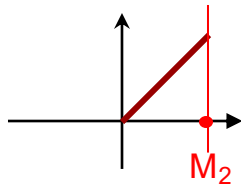
$$\hat{z}_j^{(i)} \geq 0$$



When $p_j^{(i)} = 0$:



When $p_j^{(i)} = 1$:



$$\hat{z}_j^{(i)} \leq z_j^{(i)} - M_1 \quad \hat{z}_j^{(i)} \leq 0$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)} \quad \hat{z}_j^{(i)} \geq 0$$

$$\hat{z}_j^{(i)} \leq z_j^{(i)} \quad \hat{z}_j^{(i)} \leq M_2$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)} \quad \hat{z}_j^{(i)} \geq 0$$

M_1 needs to be small (negative) enough

M_2 needs to be large enough

Pre-activation bounds

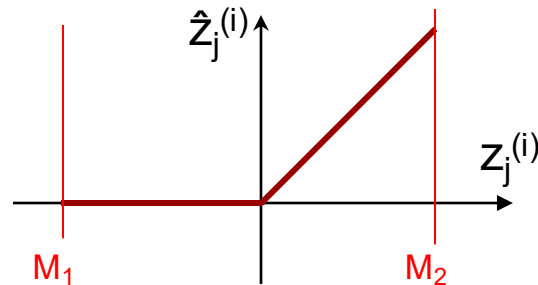
For this to work, M_1 and M_2 must be properly selected.

If set too conservatively, like $M_1 = -100000$ and $M_2 = 100000$, too many constraints will be “active” slowing the solver

How to find appropriate M_1 and M_2 ?

We can use optimization to find these pre-activation bounds - the same formulation as our verification problem before, just changing the optimization variables

MILP, LP, or more efficient methods can be used



$$\hat{z}_j^{(i)} \leq z_j^{(i)} - M_1 (1 - p_j^{(i)})$$

$$\hat{z}_j^{(i)} \leq M_2 p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$

$$l_j^{(i)} = \min z_j^{(i)}$$

$$u_j^{(i)} = \max z_j^{(i)}$$

Putting it all together with Pre-activation bounds

$$\begin{aligned} & \min y \\ \text{s.t. } & y = W^{(3)T} \hat{z}^{(2)} \\ & \hat{z}^{(2)} = \max(z^{(2)}, 0) \\ & z^{(2)} = W^{(2)} \hat{z}^{(1)} \\ & \hat{z}^{(1)} = \max(z^{(1)}, 0) \\ & z^{(1)} = W^{(1)} x \\ & x_i \leq u_i \\ & x_i \geq l_i \end{aligned}$$
$$\begin{aligned} l_j^{(i)} &= \min z_j^{(i)} \\ u_j^{(i)} &= \max z_j^{(i)} \end{aligned}$$

We can use optimization to find these pre-activation bounds - the same formulation as our verification problem before, just changing the optimization variables.

Putting it all together with Pre-activation bounds

$$\begin{aligned}
 & \min y \\
 \text{s.t. } & y = W^{(3)T} \hat{z}^{(2)} \\
 & \hat{z}^{(2)} = \max(z^{(2)}, 0) \\
 & z^{(2)} = W^{(2)} \hat{z}^{(1)} \\
 & \hat{z}^{(1)} = \max(z^{(1)}, 0) \\
 & z^{(1)} = W^{(1)} x \\
 & x_i \leq u_i^{(0)} \\
 & x_i \geq l_i^{(0)}
 \end{aligned}$$

Each ReLU is represented as

$$\begin{aligned}
 l_j^{(i)} &= \min z_j^{(i)} \\
 u_j^{(i)} &= \max z_j^{(i)} \\
 \hat{z}_j^{(i)} &\leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)}) \\
 \hat{z}_j^{(i)} &\leq u_j^{(i)} p_j^{(i)} \\
 \hat{z}_j^{(i)} &\geq z_j^{(i)} \\
 \hat{z}_j^{(i)} &\geq 0 \\
 p_j^{(i)} &\in \{0, 1\}
 \end{aligned}$$

We can use optimization to find these pre-activation bounds - the same formulation as our verification problem before, just changing the optimization variables.

Putting it all together with Pre-activation bounds

$$\begin{aligned} & \min y \\ \text{s.t. } & y = W^{(3)T} \hat{z}^{(2)} \\ & \hat{z}^{(2)} = \max(z^{(2)}, 0) \\ & z^{(2)} = W^{(2)} \hat{z}^{(1)} \\ & \hat{z}^{(1)} = \max(z^{(1)}, 0) \\ & z^{(1)} = W^{(1)} x \\ & x_i \leq u_i^{(0)} \\ & x_i \geq l_i^{(0)} \end{aligned}$$

Each ReLU is represented as

$$\begin{aligned} l_j^{(i)} &= \min z_j^{(i)} \\ u_j^{(i)} &= \max z_j^{(i)} \end{aligned}$$

$$\begin{aligned} \hat{z}_j^{(i)} &\leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)}) \\ \hat{z}_j^{(i)} &\leq u_j^{(i)} p_j^{(i)} \\ \hat{z}_j^{(i)} &\geq z_j^{(i)} \\ \hat{z}_j^{(i)} &\geq 0 \\ p_j^{(i)} &\in \{0, 1\} \end{aligned}$$

Integer variables are still hard!

Putting it all together with Pre-activation bounds

$$\begin{aligned} y_{MILP}^* &= \min y \\ \text{s.t. } y &= W^{(3)T} \hat{z}^{(2)} \\ \hat{z}^{(2)} &= \max(z^{(2)}, 0) \\ z^{(2)} &= W^{(2)} \hat{z}^{(1)} \\ \hat{z}^{(1)} &= \max(z^{(1)}, 0) \\ z^{(1)} &= W^{(1)} x \\ x_i &\leq u_i^{(0)} \\ x_i &\geq l_i^{(0)} \end{aligned}$$

Integer variables are still hard!

Each ReLU is represented as

$$\begin{aligned} l_j^{(i)} &= \min z_j^{(i)} \\ u_j^{(i)} &= \max z_j^{(i)} \end{aligned}$$

$$\begin{aligned} \hat{z}_j^{(i)} &\leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)}) \\ \hat{z}_j^{(i)} &\leq u_j^{(i)} p_j^{(i)} \\ \hat{z}_j^{(i)} &\geq z_j^{(i)} \\ \hat{z}_j^{(i)} &\geq 0 \\ p_j^{(i)} &\in \{0, 1\} \end{aligned}$$

Putting it all together with Pre-activation bounds

$$\begin{aligned}
 & y_{LP}^* = \min y \\
 & s.t. \quad y = W^{(3)T} \hat{z}^{(2)} \\
 & \quad \hat{z}^{(2)} = \max(z^{(2)}, 0) \\
 & \quad z^{(2)} = W^{(2)} \hat{z}^{(1)} \\
 & \quad \hat{z}^{(1)} = \max(z^{(1)}, 0) \\
 & \quad z^{(1)} = W^{(1)} x \\
 & \quad x_i \leq u_i^{(0)} \\
 & \quad x_i \geq l_i^{(0)}
 \end{aligned}$$

Linear relaxation of integer constraints

How does this affect the solution?

Each ReLU is represented as

$$\begin{aligned}
 l_j^{(i)} &= \min z_j^{(i)} \\
 u_j^{(i)} &= \max z_j^{(i)}
 \end{aligned}$$

$$\begin{aligned}
 \hat{z}_j^{(i)} &\leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)}) \\
 \hat{z}_j^{(i)} &\leq u_j^{(i)} p_j^{(i)} \\
 \hat{z}_j^{(i)} &\geq z_j^{(i)} \\
 \hat{z}_j^{(i)} &\geq 0 \\
 0 &\leq p_j^{(i)} \leq 1
 \end{aligned}$$

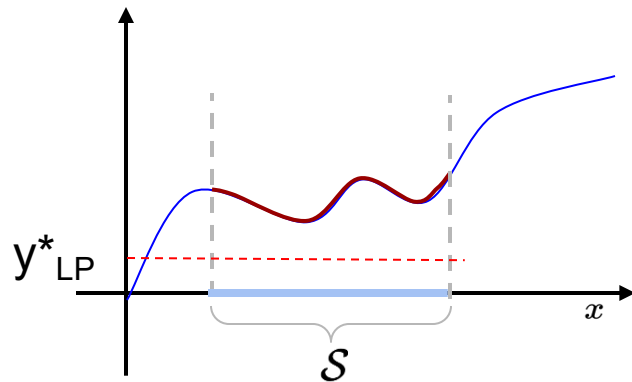
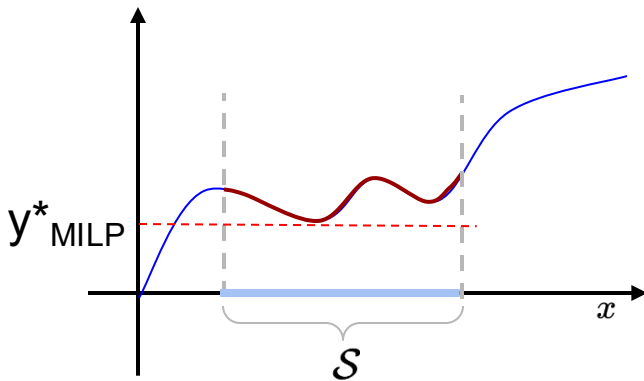
MILP vs LP

$$y^*_{LP} \leq y^*_{MILP}$$

It's not the original MILP solution, but it is a guaranteed lower bound

Solving LP is (weakly) polynomial time.

Simplex algorithm practically a few orders of magnitude faster

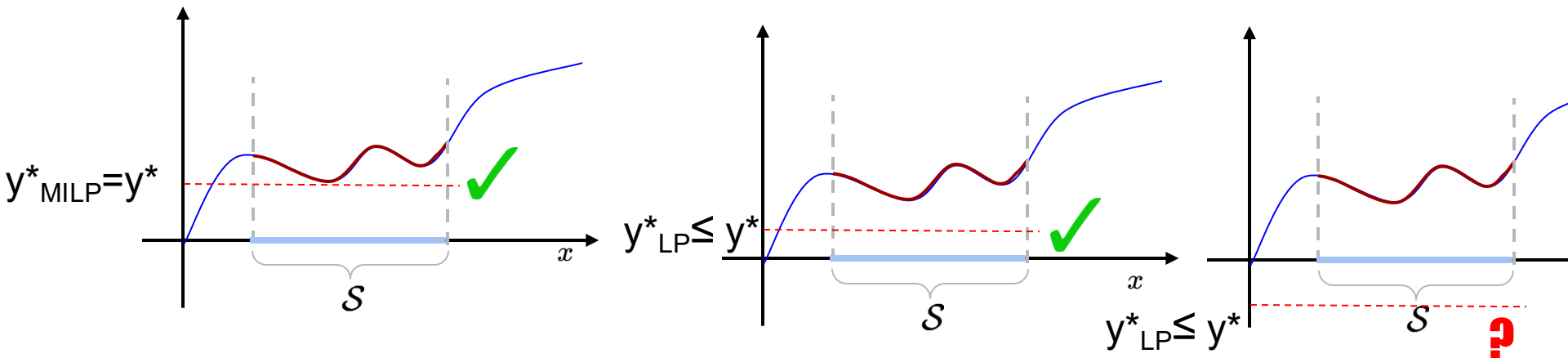


Sound but Incomplete Verification with a Lower Bound

Satisfiability problem: $\exists x \in S \wedge y \leq 0 \wedge y = f(x)$

If $y^*_{LP} > 0 \Rightarrow y^* > 0 \Rightarrow$ requirement verified (unsatisfiable)

$y^*_{LP} \leq 0 \Rightarrow$ verification result unknown (incomplete)



A closer look at the LP relaxation: “Triangle” relaxation

Each ReLU is represented as

$$l_j^{(i)} = \min z_j^{(i)}$$
$$u_j^{(i)} = \max z_j^{(i)}$$

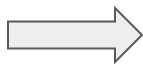
$$\hat{z}_j^{(i)} \leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)})$$

$$\hat{z}_j^{(i)} \leq u_j^{(i)} p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

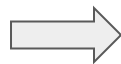
$$\hat{z}_j^{(i)} \geq 0$$

$$0 \leq p_j^{(i)} \leq 1$$



$$p_j^{(i)} \leq \frac{\hat{z}_j^{(i)} - z_j^{(i)} + l_j^{(i)}}{l_j^{(i)}}$$

$$p_j^{(i)} \geq \frac{\hat{z}_j^{(i)}}{u_j^{(i)}}$$



Project out p

$$\hat{z}_j^{(i)} \leq \frac{u_j^{(i)}}{u_j^{(i)} - l_j^{(i)}} z_j^{(i)} - \frac{u_j^{(i)} l_j^{(i)}}{u_j^{(i)} - l_j^{(i)}}$$

(Please note that $l_j^{(i)}$ is negative during the above derivation)

A closer look at the linear programming relaxation

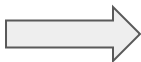
Each ReLU is represented by

$$\hat{z}_j^{(i)} \leq z_j^{(i)} - l_j^{(i)}(1 - p_j^{(i)})$$

$$\hat{z}_j^{(i)} \leq u_j^{(i)} p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$

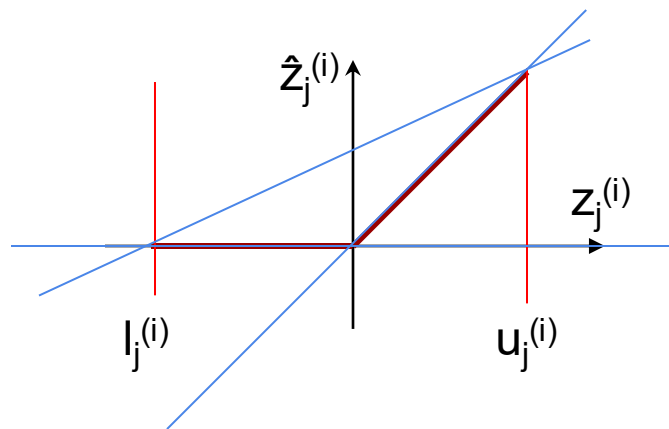


$$\hat{z}_j^{(i)} \leq \frac{u_j^{(i)}}{u_j^{(i)} - l_j^{(i)}} z_j^{(i)} - \frac{u_j^{(i)} l_j^{(i)}}{u_j^{(i)} - l_j^{(i)}}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$

“Triangle” relaxation



A closer look at the linear programming relaxation

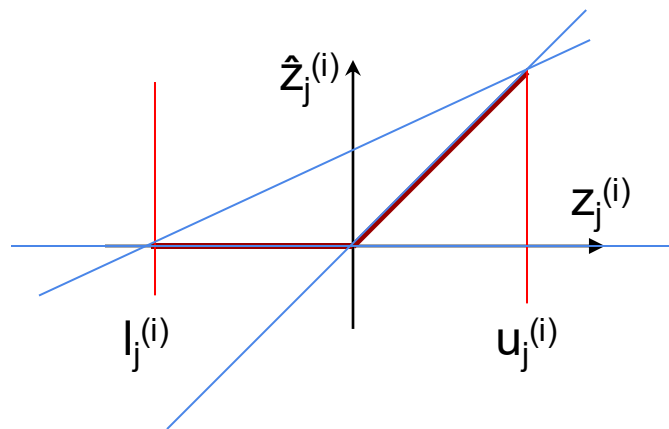
MILP: solutions are constrained on **ReLU function**

Linear programming: solutions are constrained on the **triangle**

$$\hat{z}_j^{(i)} \leq \frac{u_j^{(i)}}{u_j^{(i)} - l_j^{(i)}} z_j^{(i)} - \frac{u_j^{(i)} l_j^{(i)}}{u_j^{(i)} - l_j^{(i)}}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$

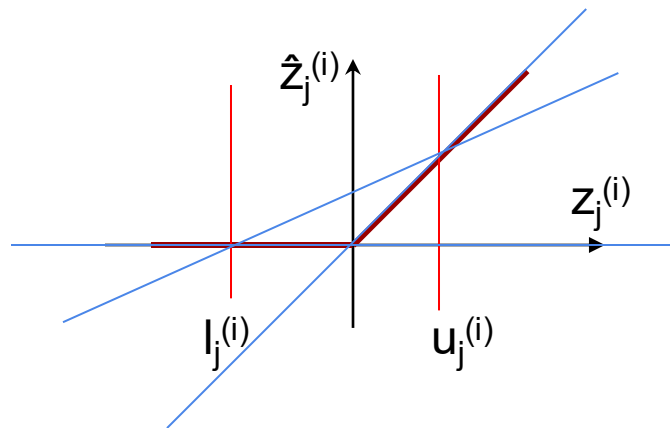
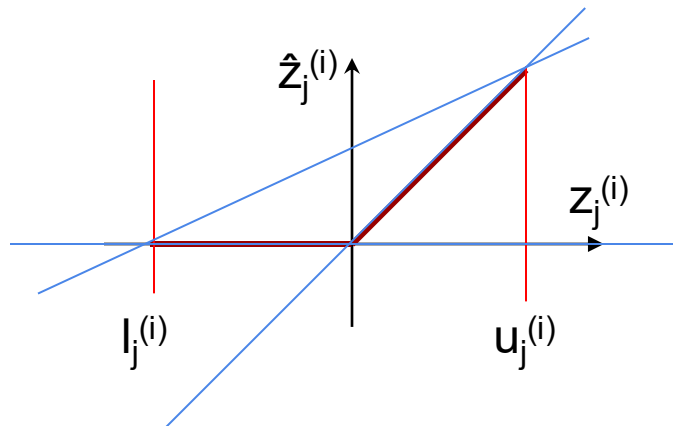


A closer look at the linear programming relaxation

MILP: solutions are constrained on **ReLU function**

Linear programming: solutions are constrained on the **triangle**

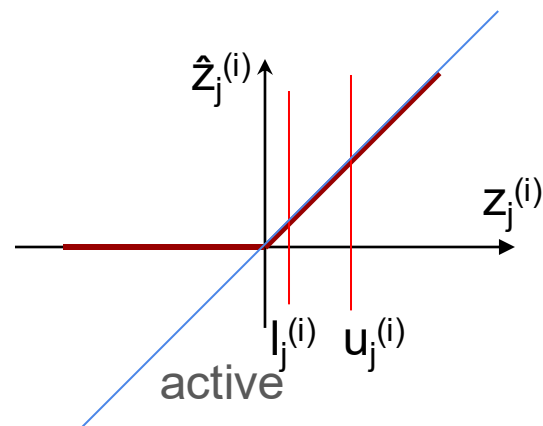
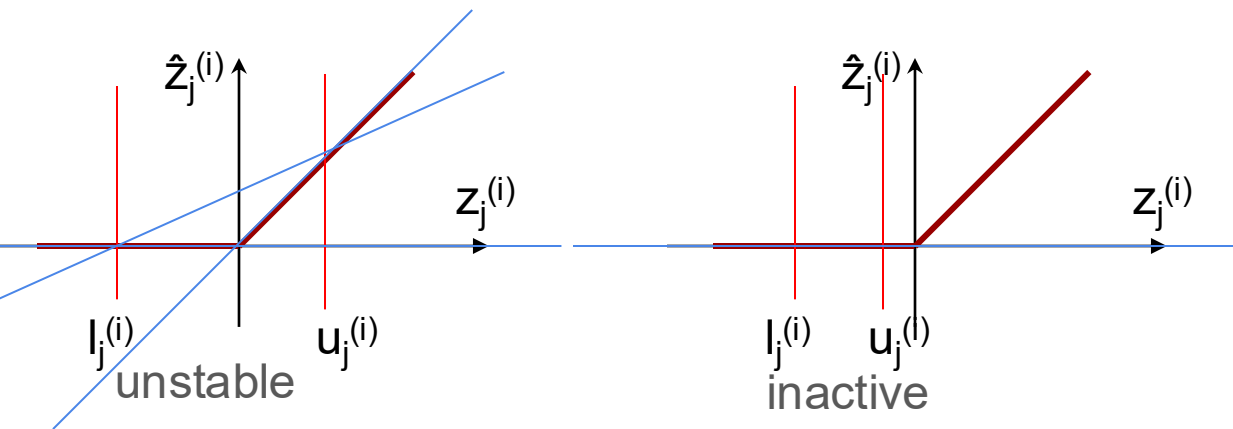
We want the triangle to be as small as possible! So tight pre-activation bounds are necessary.



Stable vs. unstable neurons

If bounds are tight enough so $l_j^{(i)} \geq 0$ or $u_j^{(i)} \leq 0$, it is called stable neurons (active or inactive); otherwise it is a unstable neuron

In stable neuron cases, a binary variable in MILP or relaxation in LP is not needed - ReLU becomes a linear function.



Stable vs. unstable neurons

$$\hat{z}_j^{(i)} \leq z_j^{(i)} - l_j^{(i)} (1 - p_j^{(i)})$$

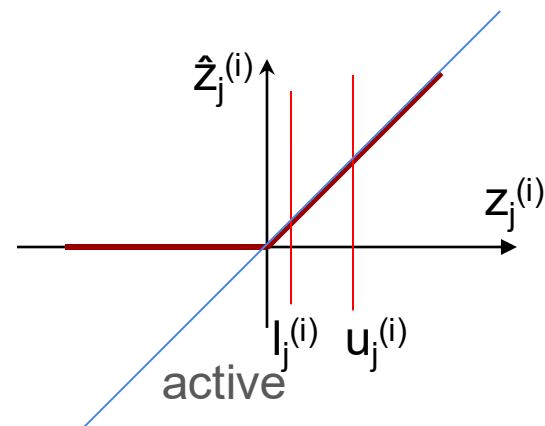
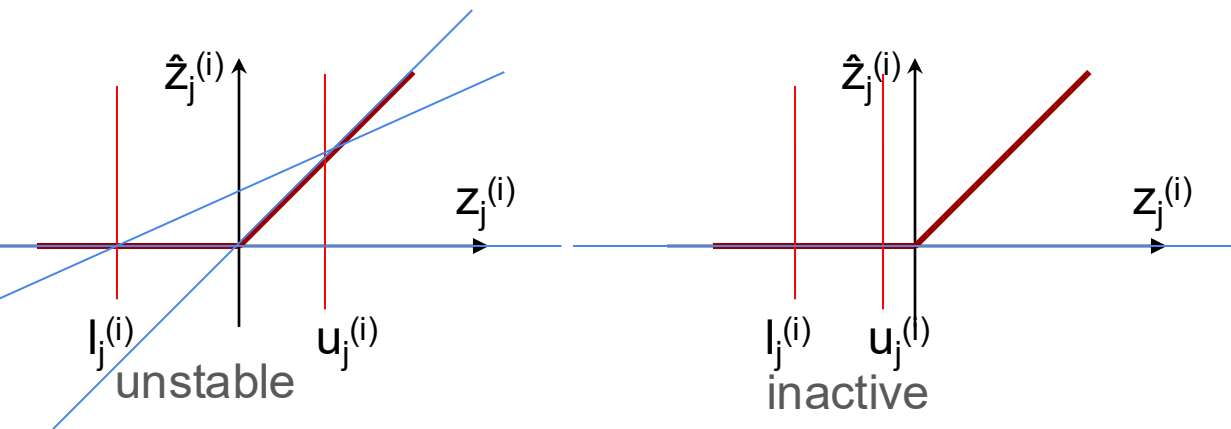
$$\hat{z}_j^{(i)} \leq u_j^{(i)} p_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq z_j^{(i)}$$

$$\hat{z}_j^{(i)} \geq 0$$

$$\hat{z}_j^{(i)} = 0$$

$$\hat{z}_j^{(i)} = z_j^{(i)}$$



Summary: MILP vs LP vs DPLL(T) in Neural Network Verification

MILP:

- Branch and bound is used to make decisions only on certain number of binary variables
- No decision needed on stable neurons (with the help of pre-activation bounds)
- Specialized methods to accelerate solving (e.g., branching heuristics, cutting planes)
- Complete (solve y^* exactly)
- Typically scales much better than DPLL(T)

Linear programming:

- No integer variables
- No variable decisions needed (no exponential time search)
- Simplex algorithm can solve it relatively fast, a few thousands neurons are ok
- Incomplete (has to return unknown in some cases)

Next: more efficient algorithms for NN verification

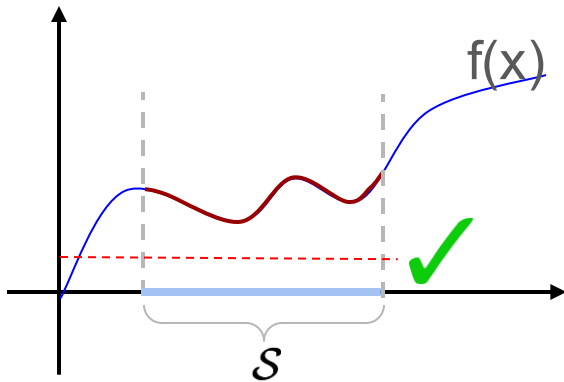
Solving neural network verification using SMT solvers

Solving neural network verification using optimization (MIP/LP)

Solving neural network verification using **bound propagation**

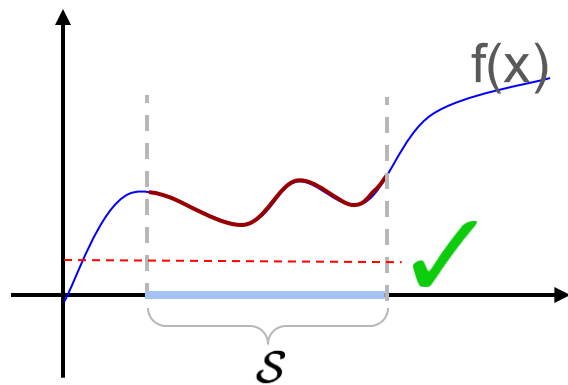
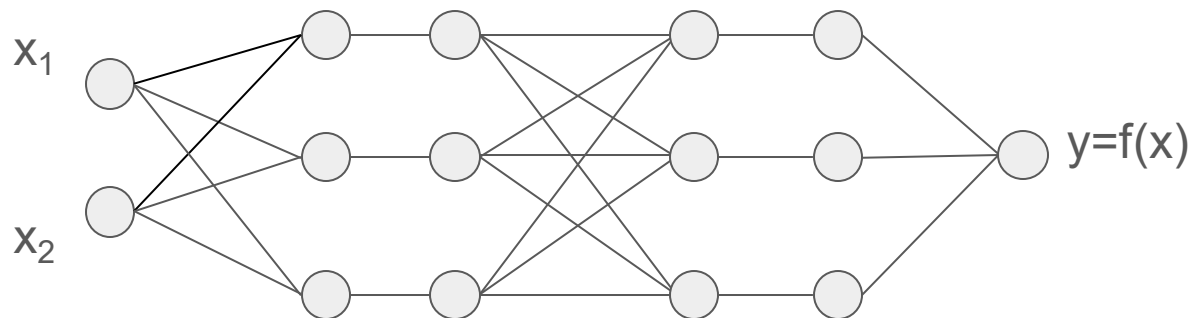
- Interval bound propagation (IBP)
- Linear (symbolic) bound propagation (CROWN)

Efficient methods are typically incomplete (solving a lower bound, as tight as possible)

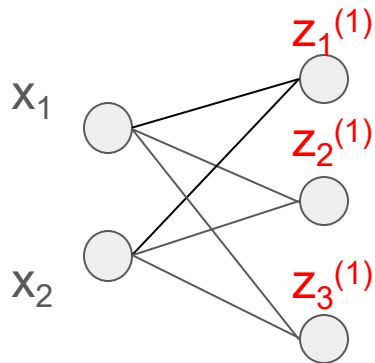


$$y^* = \min_{x \in S} f(x)$$

Any faster ways to calculate the bounds on $f(x)$?



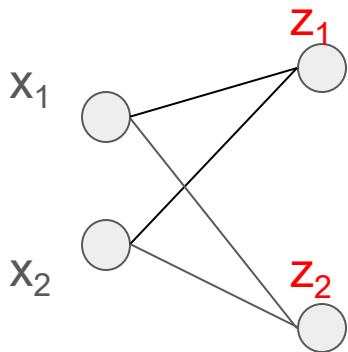
Let's look at one layer first



Given bounds on x , can we calculate the bounds on z ?

$$x_1 \in [-1, 2], \quad x_2 \in [-2, 1]$$

Let's look at one layer first



Given bounds on x , can we calculate the bounds on z ?

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

As an illustration, suppose we have

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

Can you infer bounds on z given bounds on x ?

Interval Bound Propagation (IBP)

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

Lower bounds

$$\underline{z}_1 = -1 - 1 = -2$$

$$\underline{z}_2 = -1 \times 2 - 1 = -3$$

Upper bounds

$$\bar{z}_1 = 2 - (-2) = 4$$

$$\bar{z}_2 = 2 \times 2 - (-2) = 6$$

Interval Bound Propagation (IBP)

$$x_1 \in [-1, 2], x_2 \in [-2, 1] \quad z_1 = x_1 - x_2 \quad z_2 = 2x_1 - x_2$$

$$\underline{z}_1 = -1 - 1 = -2 \quad \bar{z}_1 = 2 - (-2) = 4$$

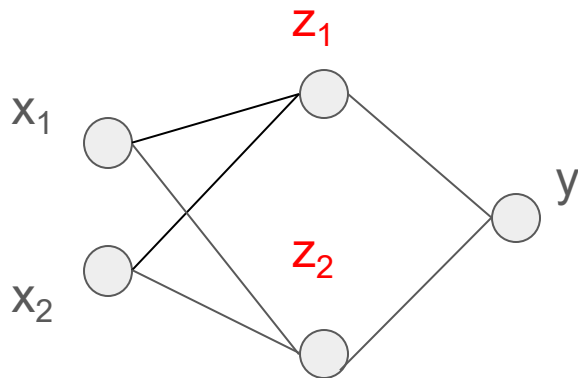
$$\underline{z}_2 = -1 \times 2 - 1 = -3 \quad \bar{z}_2 = 2 \times 2 - (-2) = 6$$

In general:

$$\sum_{i \in \{i | w_i \geq 0\}} w_i \underset{\uparrow}{l_i} + \sum_{i \in \{i | w_i < 0\}} w_i \underset{\uparrow}{u_i} \leq \sum_i w_i x_i \leq \sum_{i \in \{i | w_i \geq 0\}} w_i u_i + \sum_{i \in \{i | w_i < 0\}} w_i l_i$$

Elements lower and upper bounds of x

Interval Bound Propagation: continue to the next layer



Let's say $y = z_1 - z_2$

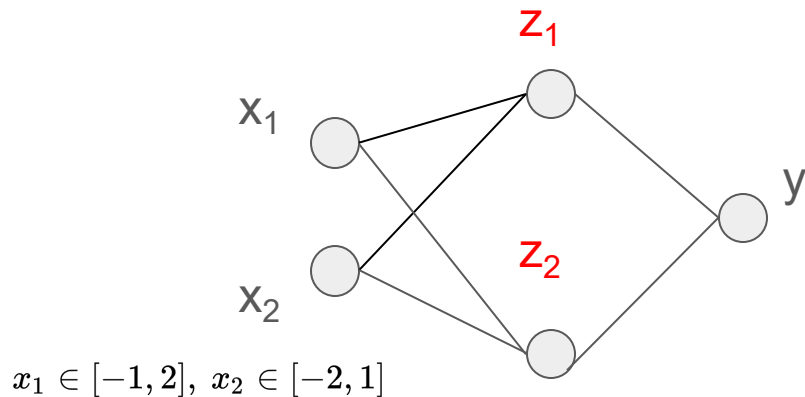
We also know that:

$$z_1 \in [-2, 4] \quad z_2 \in [-3, 6]$$

The what can we conclude about y ?

$$y \in [-8, 7]$$

Interval Bound Propagation: limitations



Apply IBP we obtain $y \in [-8, 7]$
for this simple linear network.

However observe that

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

$$y = z_1 - z_2$$

$$y = x_1 - x_2 - (2x_1 - x_2) = -x_1$$

The actual bounds is $[-2, 1]$, much tighter than $[-8, 7]$

A Better Idea: Keep the correlations between x and z

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

$$y = z_1 - z_2$$

$$y = x_1 - x_2 - (2x_1 - x_2) = -x_1$$

The actual bounds is $[-2, 1]$, much tighter than $[-8, 7]$

It is important to keep the correlations between z and x to obtain this tighter result!

We treat z as a **symbolic function of x** , rather than intervals

A Better Idea: linear bound propagation

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

$$y = z_1 - z_2$$

$$y = x_1 - x_2 - (2x_1 - x_2) = -x_1$$

The actual bounds is $[-2, 1]$, much tighter than $[-8, 7]$

It is important to keep the correlations between z and x to obtain this tighter result!

We treat z as a **linear function of x** , rather than intervals

A Better Idea: linear bound propagation

$$y = z_1 - z_2 \longrightarrow y = x_1 - x_2 - (2x_1 - x_2) = -x_1$$

Plug in

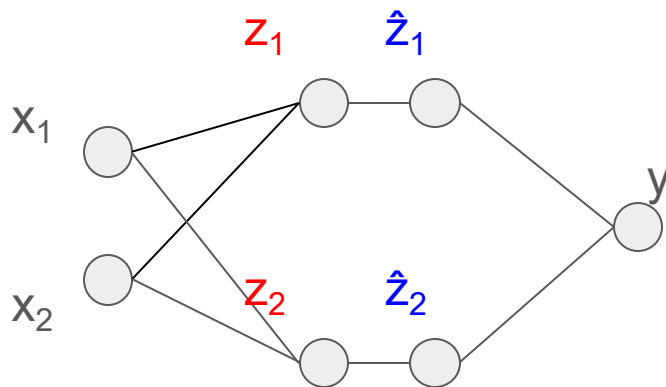
$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

We treat z as a **linear function of x** , rather than concrete intervals.

After we plug in linear functions (z w.r.t. x), we still get a linear function (y w.r.t. x)

Bound propagation: how about nonlinear functions?



Can we improve IBP using symbolic linear bounds?

Instead of $y = z_1 - z_2$

Now we have $y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$

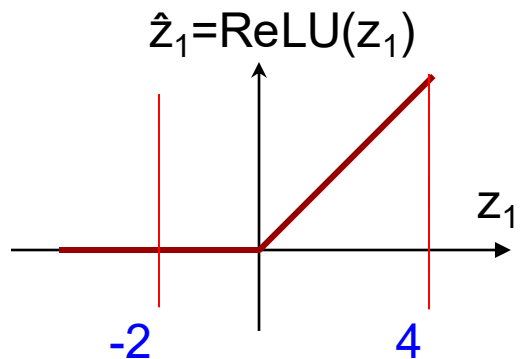
From IBP we already know that

$$z_1 \in [-2, 4], \quad z_2 \in [-3, 6],$$

$$\text{ReLU}(z_1) \in [0, 4], \quad \text{ReLU}(z_2) \in [0, 6]$$

$$y \in [-6, 4]$$

Linear bound propagation for ReLU function (CROWN)



Instead of $y = z_1 - z_2$

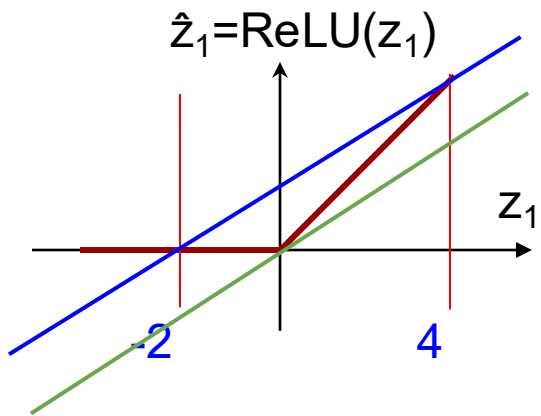
Now we have $y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$

We already know that

$$z_1 \in [-2, 4], \quad z_2 \in [-3, 6],$$

(Preactivation bounds)

Linear bound propagation for ReLU function (CROWN)

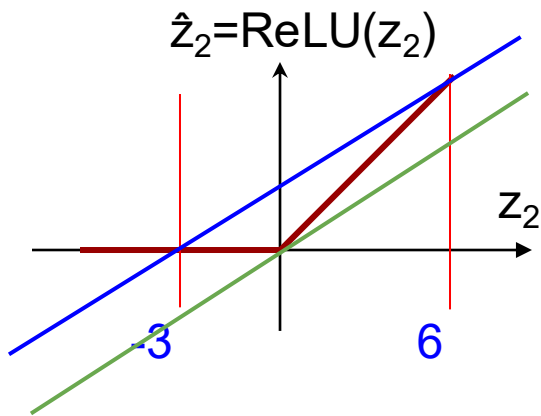


Linear **upper bound** (same as the one of triangle relaxation in LP)

Linear **lower bound** (actually not unique)

$$\boxed{\frac{2}{3} z_1} \leq \text{ReLU}(z_1) \leq \boxed{\frac{2}{3} z_1 + \frac{4}{3}}$$

Linear bound propagation for ReLU function (CROWN)



$\text{ReLU}(z_2)$ can be bounded using linear functions similarly.

Now let's consider $y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$. How to bound it using linear functions of z_1 and z_2 ?

$$\frac{2}{3}z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$\frac{2}{3}z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$

Linear bound propagation for ReLU function (CROWN)

$$\boxed{\frac{2}{3}z_1} \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$\frac{2}{3}z_2 \leq \text{ReLU}(z_2) \leq \boxed{\frac{2}{3}z_2 + 2}$$

Negative coefficient, take
upper bound

$$\boxed{\frac{2}{3}z_1} - \boxed{\left(\frac{2}{3}z_2 + 2\right)} \leq$$

positive coefficient, take
lower bound

$$y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$$

$$\leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - \frac{2}{3}z_2$$

Linear bound propagation for ReLU function (CROWN)

$$\frac{2}{3}z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - \frac{2}{3}z_2$$

Now we have linear inequalities for y w.r.t. z !

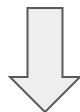
Next step we can simply plug in, as in the linear ($y=z_1-z_2$) case.

Linear bound propagation for ReLU function (CROWN)

$$\frac{2}{3}z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - \frac{2}{3}z_2$$

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

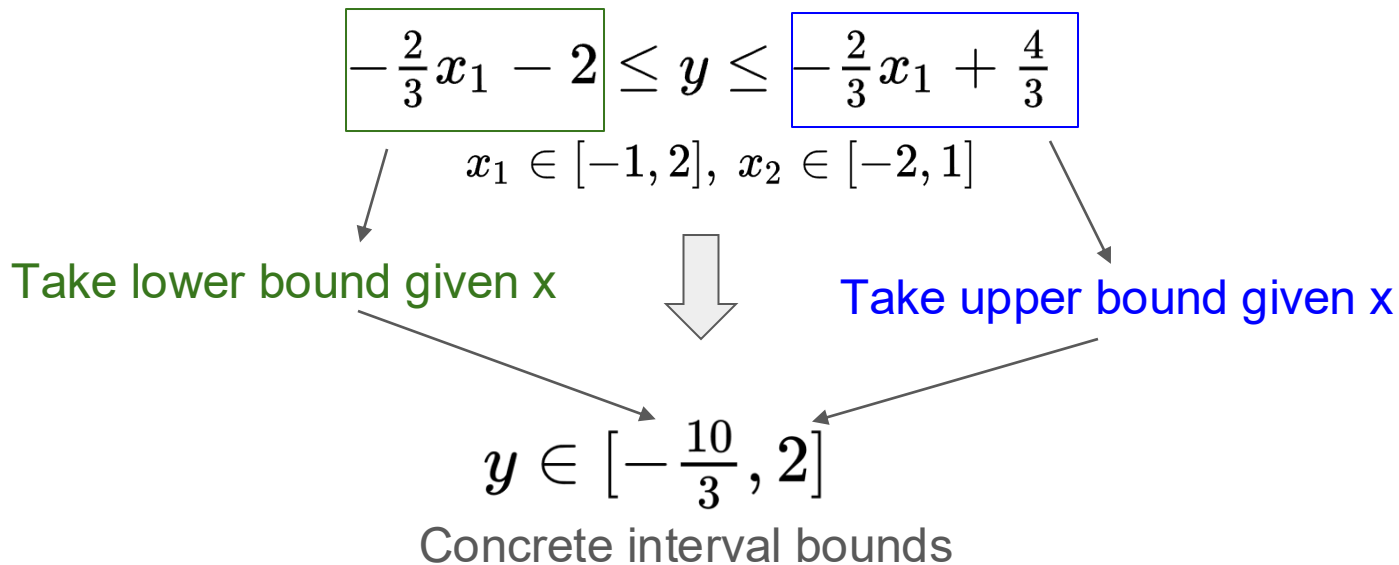


Plug in

$$-\frac{2}{3}x_1 - 2 \leq y \leq -\frac{2}{3}x_1 + \frac{4}{3}$$

Linear bound propagation for ReLU function (CROWN)

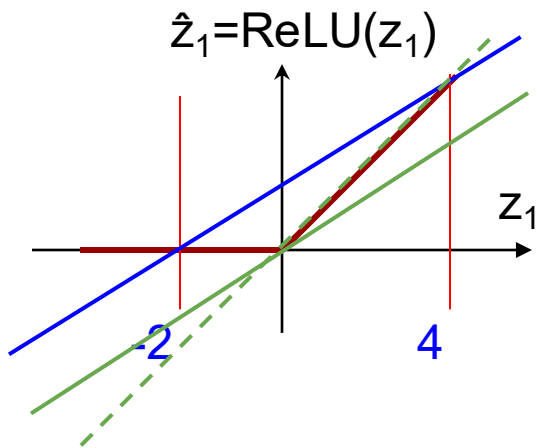
We now have symbolic linear bounds for y w.r.t. x



A lot more tighter than IBP bounds $y \in [-6, 4]$

Can we do even better?

Let's recall that when we linearly bound the ReLU function, there are some flexibilities



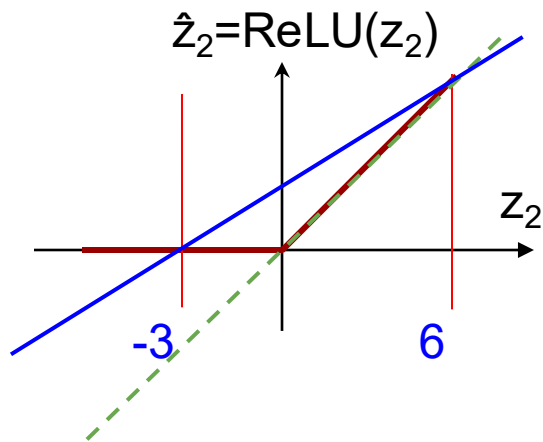
Linear **upper bound** (same as the one of triangle relaxation in LP)

Linear **lower bound** (actually **not unique**)

$$\frac{2}{3}z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

Also valid: $z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$

Choosing different linear bounds (α -CROWN)



Now what are the linear bounds of
 $y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$?

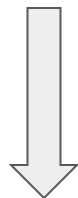
$$z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - z_2$$

$$z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$

Choosing different linear bounds (α -CROWN)

$$z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - z_2$$

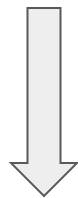


$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

Plug in

$$-\frac{1}{3}x_1 - \frac{1}{3}x_2 - 2 \leq y \leq -\frac{4}{3}x_1 + \frac{1}{3}x_2 + \frac{4}{3}$$

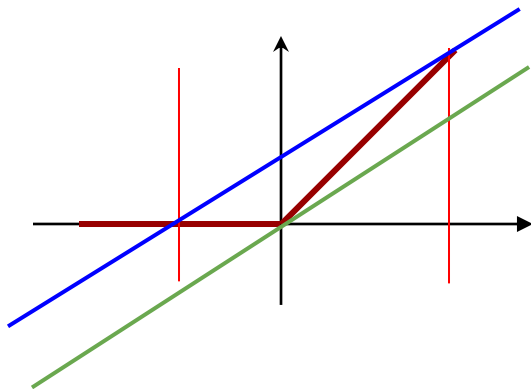


$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

Concretize

$$y \in [-3, 3]$$

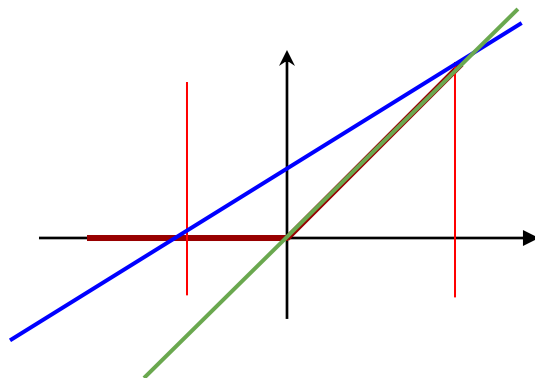
Linear lower bounds for ReLU function matters!



$$\frac{2}{3}z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$
$$\frac{2}{3}z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$



$$y \in \left[-\frac{10}{3}, 2\right]$$



$$z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$
$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$



$$y \in [-3, 3]$$

Which one is correct?

Linear lower bounds for ReLU function matters!

Both results are correct! But we want the bounds to be as tight as possible! So best result is $y \in [-3, 2]$

In general, the slope of the linear lower bound for every ReLU neuron can be optimized to find the best result.

Linear lower bounds for ReLU function matters!

In general, the slope of the linear lower bound for every ReLU neuron can be optimized to find the best result.

$$\alpha_1 z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3} z_2 + \frac{4}{3}$$

$$\alpha_2 z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3} z_2 + 2$$

For optimal lower bound of y , set $\alpha_1=1$, $\alpha_2=1$

For optimal upper bound of y , set $\alpha_1=2/3$, $\alpha_2=2/3$

(note that the optimal α_1 and α_2 do not equal in general)

Linear bound propagation method (CROWN)

1. Obtain all pre-activation bounds (can be done via CROWN recursively)
2. Start from the output layer, form the initial linear (in)equality $y = y$
3. Recursively propagate linear inequality $y \leq a^T z + b$ through each layer:
 - a. For a linear layer, $z = Wz'$, directly plug in $a^T z + b$ to get a linear bound of z'
 - b. For a non-linear layer (e.g., $z = \text{ReLU}(z')$), we first form the linear inequalities to bound the nonlinear layer itself. Then multiply either the lower or upper bound based on the sign of element in a
4. When the linear inequality propagates to the input layer, we can concretize the linear bound using bounds on input layer.

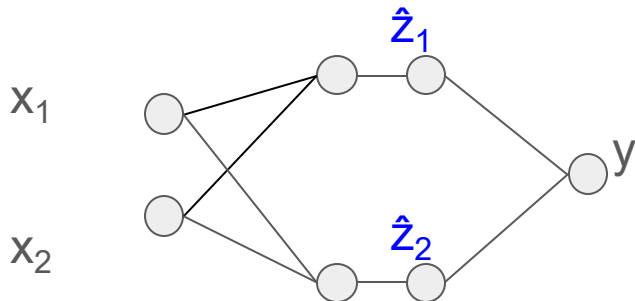
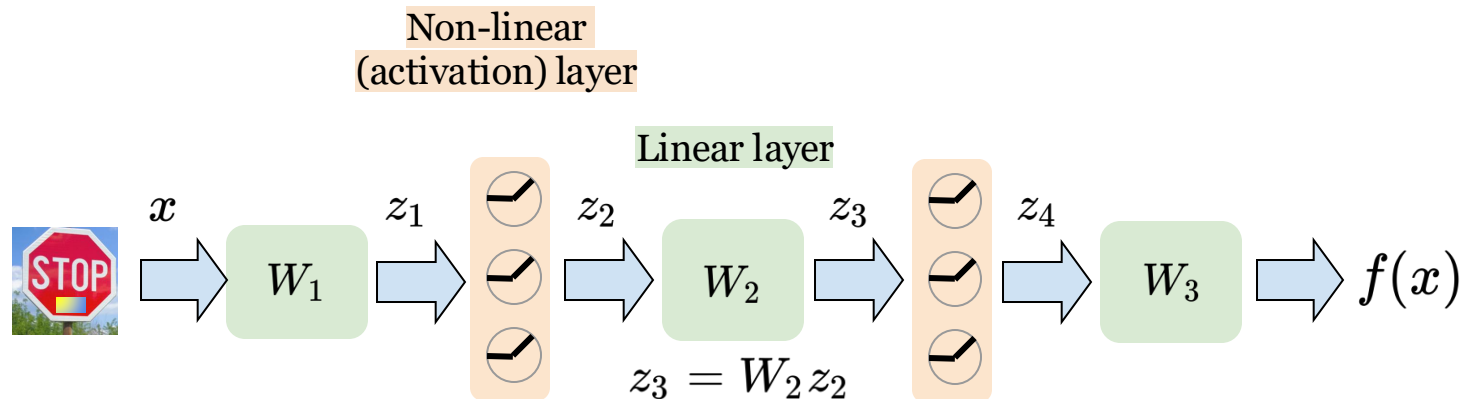
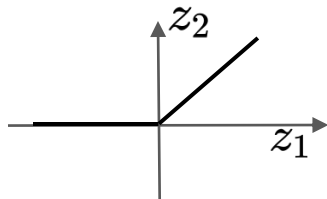


Illustration: Linear bound propagation process



$$z_2 = \text{ReLU}(z_1)$$

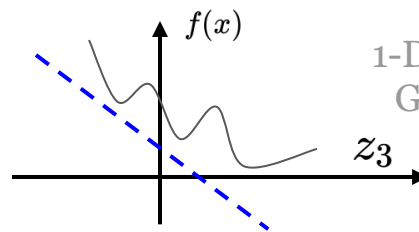


Steps:

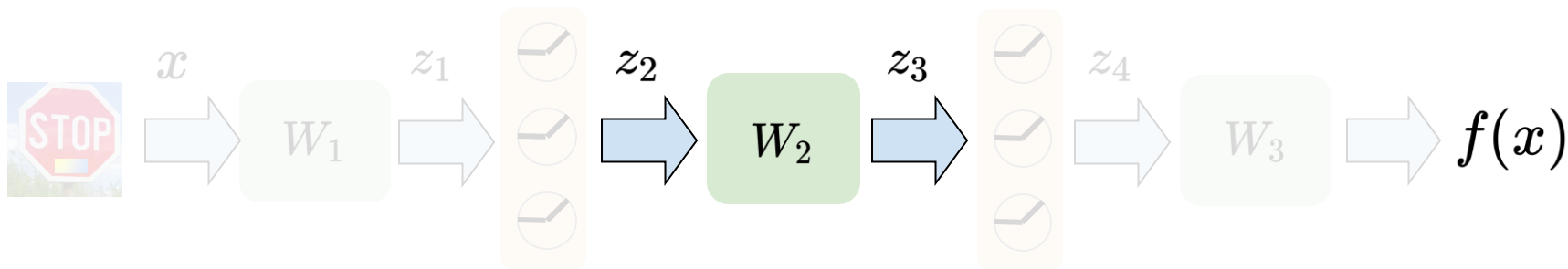
- Propagate bounds through linear layers
- Propagate bounds through non-linear layers

Illustration: Linear bound propagation process

A linear lower bound for an intermediate layer

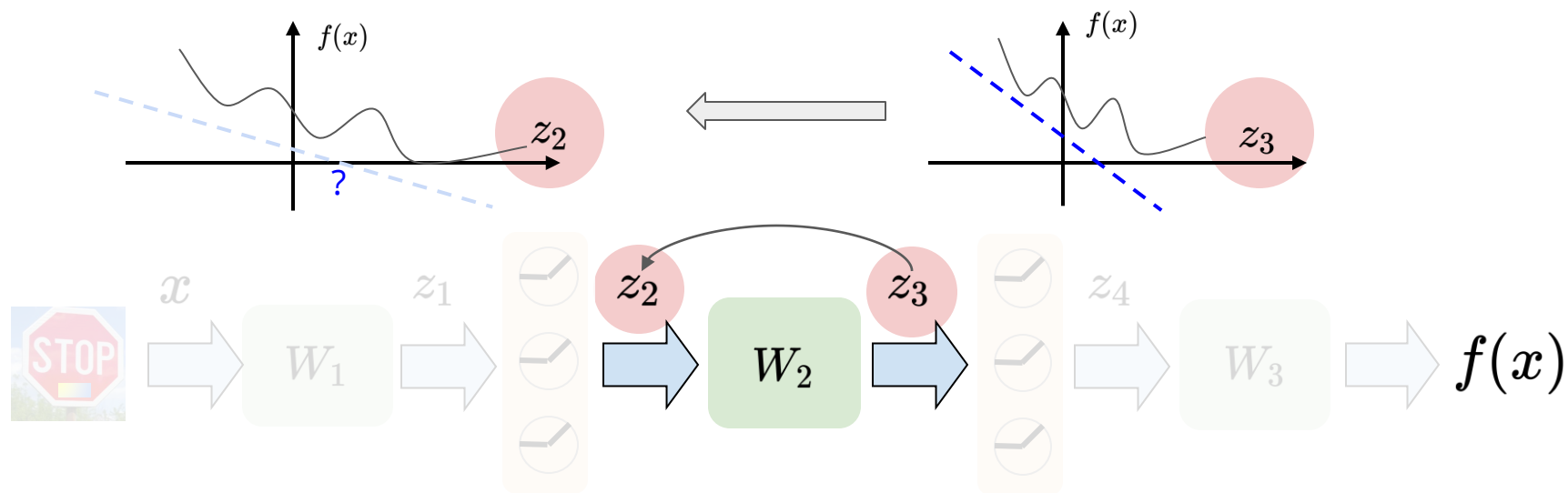


1-D case for illustration.
Generally it's a linear
hyperplane



$$f(x) \geq a^\top z_3 + b$$

Illustration: Linear bound propagation process



Propagate it to one layer before,
while keeping the lower bound valid

Illustration: Linear bound propagation process

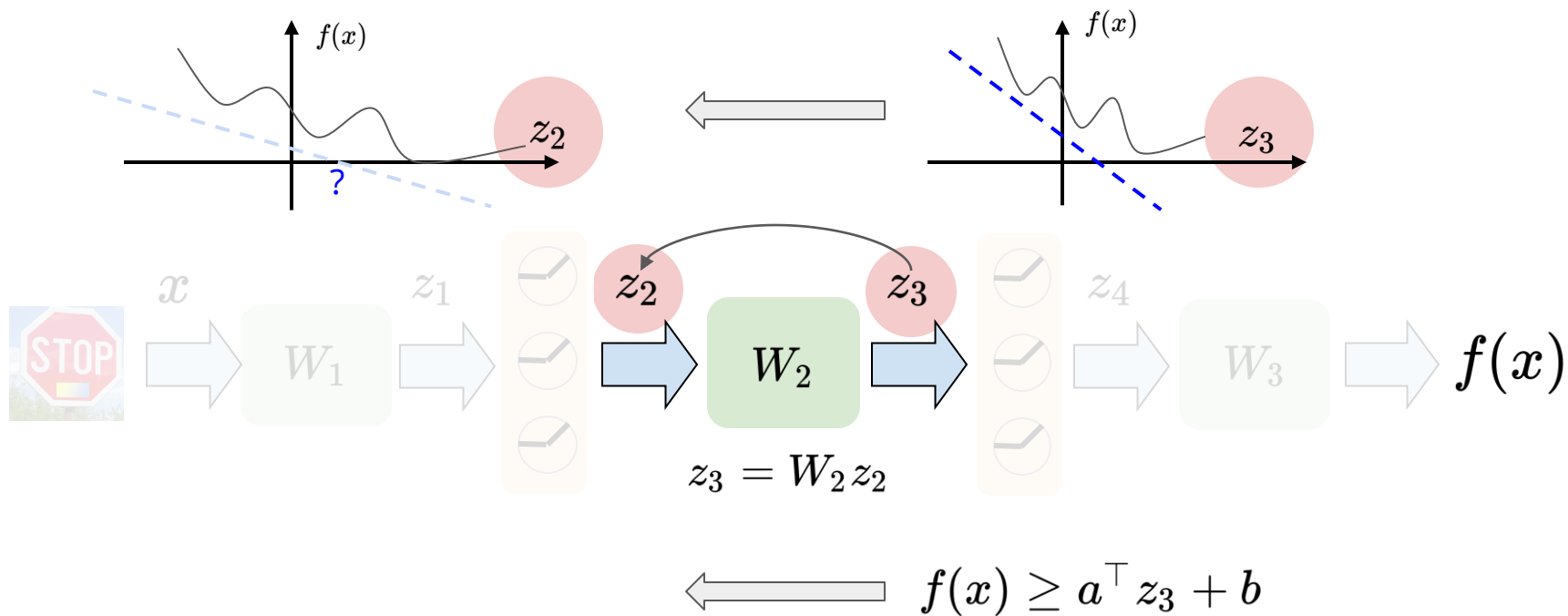


Illustration: Linear bound propagation process

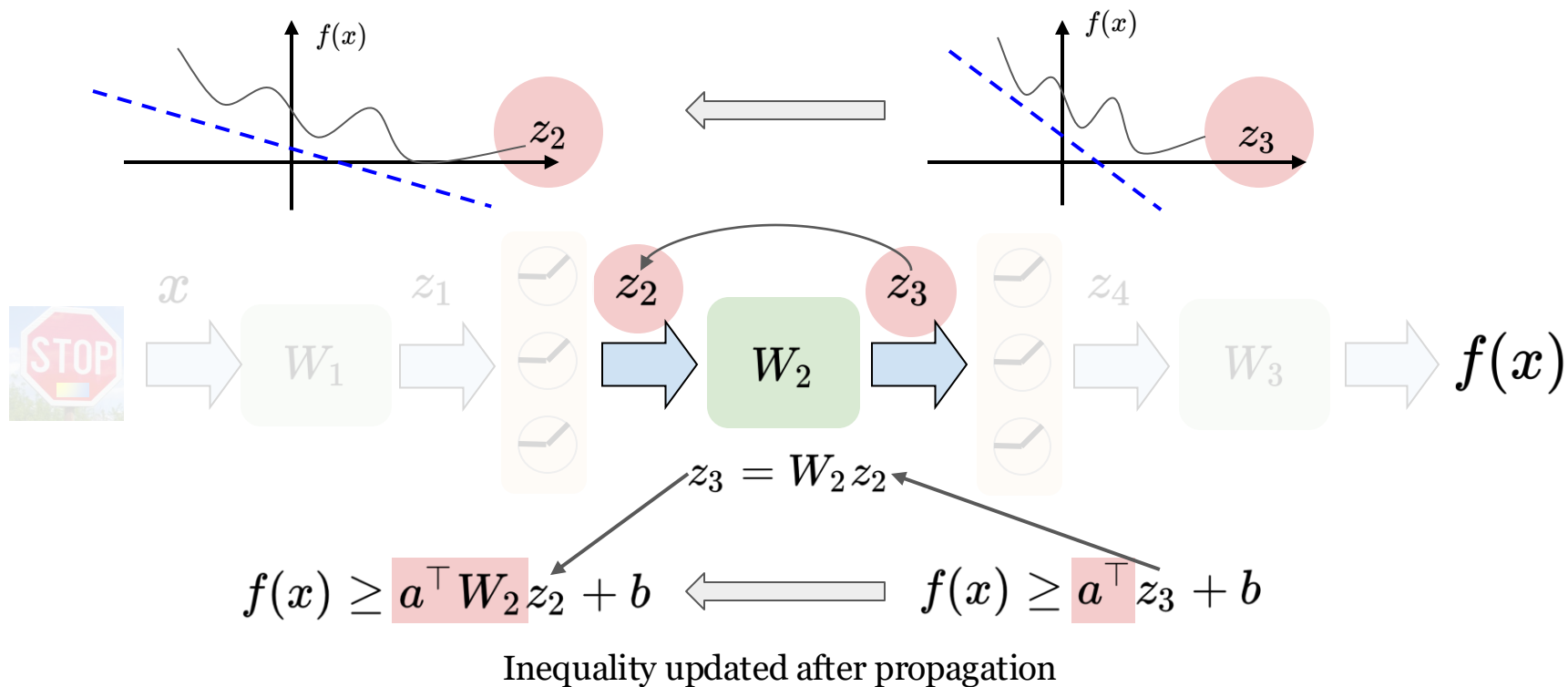


Illustration: Linear bound propagation process

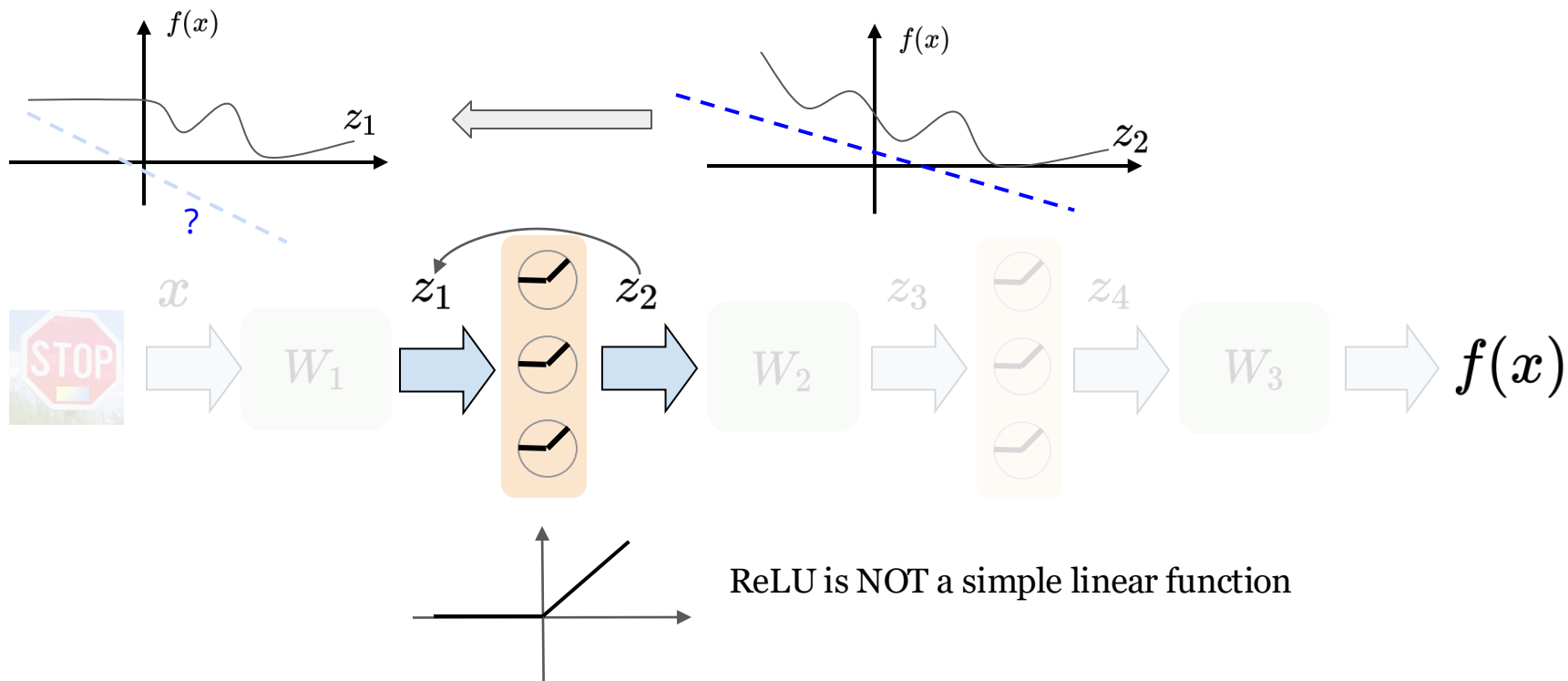
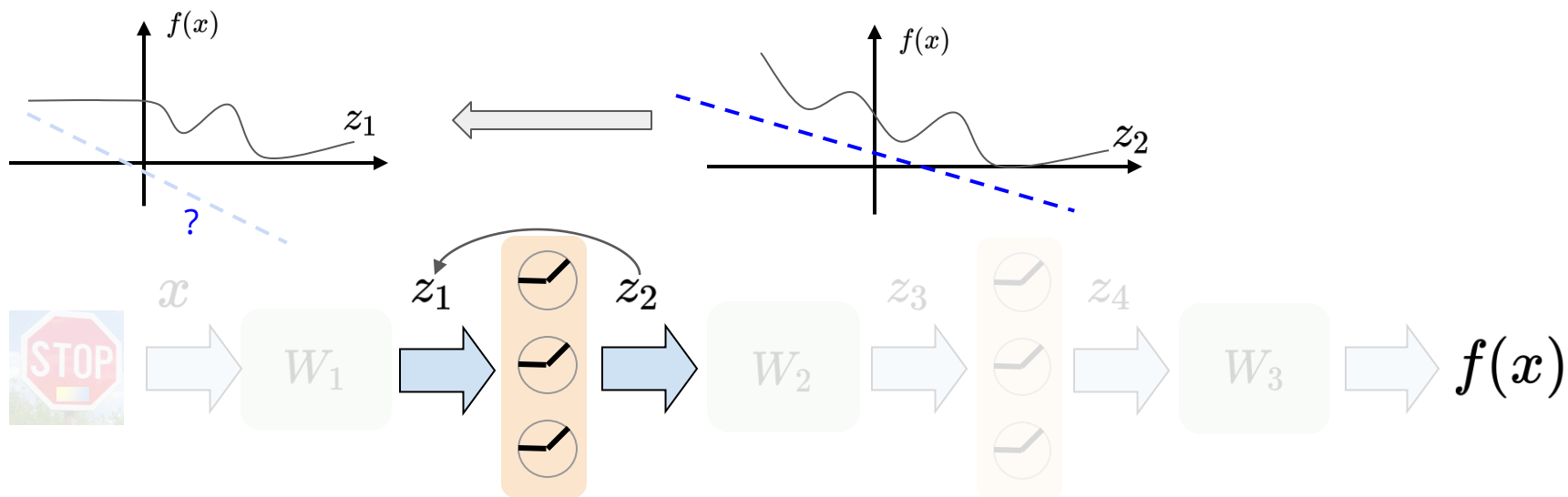
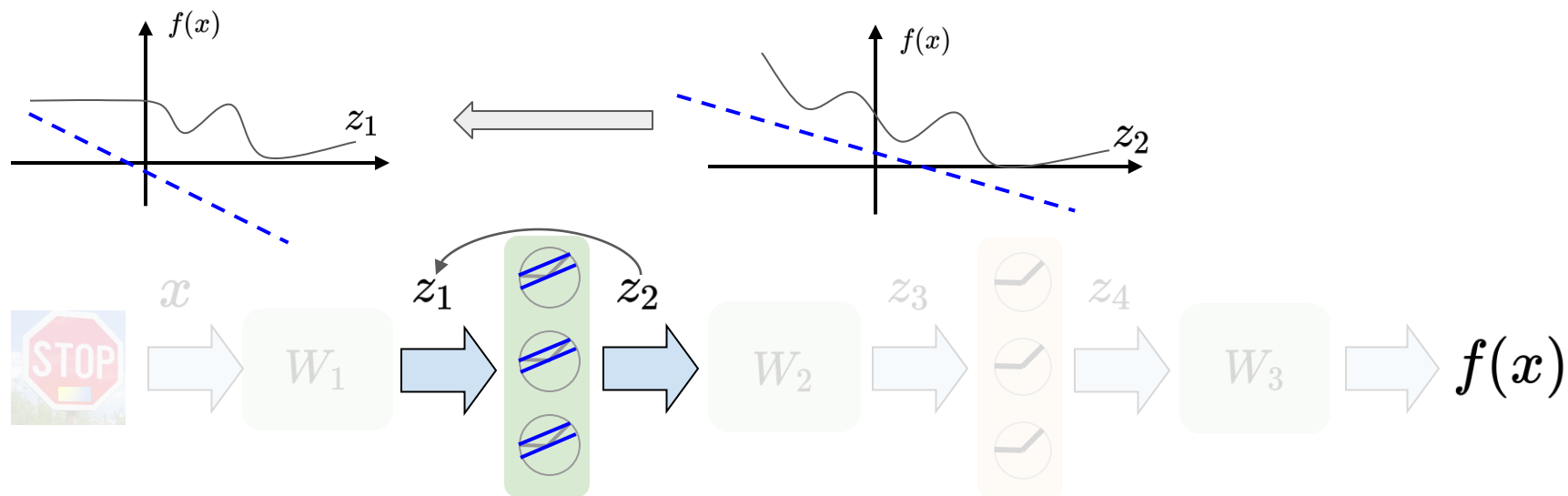


Illustration: Linear bound propagation process



$$\longleftarrow f(x) \geq a^\top W_2 z_2 + b \quad \forall x \in \mathcal{S}$$

Illustration: Linear bound propagation process

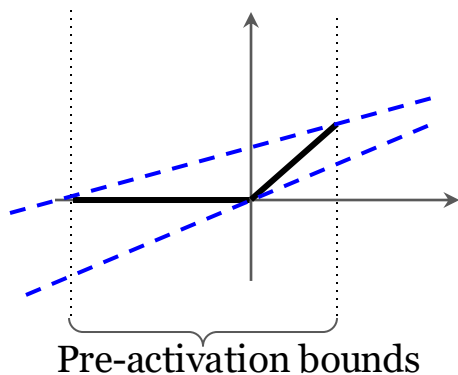


Theorem (informal): we can efficiently find D , b' such that:

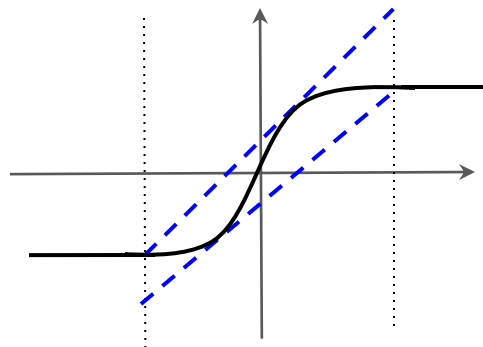
$$f(x) \geq a^\top W_2 D z_1 + b' \longleftarrow f(x) \geq a^\top W_2 z_2 + b \quad \forall x \in \mathcal{S}$$

Illustration: Linear bound propagation process

Proof sketch: conservatively use linear bounds to replace a non-linear function.



(can be pre-computed using CROWN)

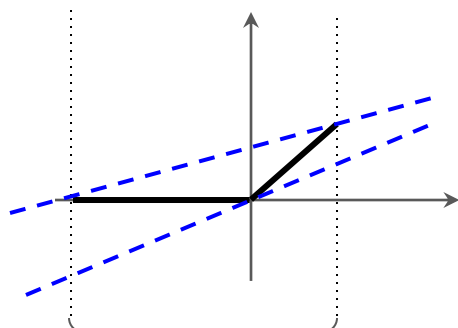


Theorem (informal): we can efficiently find D , b' such that:

$$f(x) \geq a^\top W_2 D z_1 + b' \iff f(x) \geq a^\top W_2 z_2 + b \quad \forall x \in \mathcal{S}$$

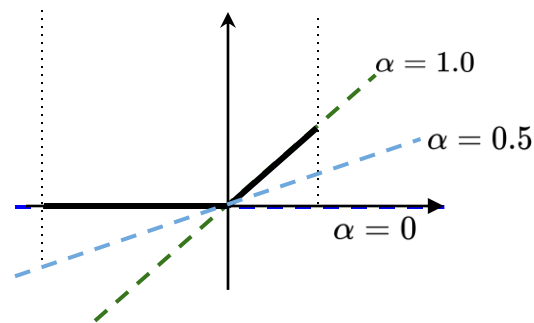
Illustration: Linear bound propagation process

Proof sketch: conservatively use linear bounds to replace a non-linear function.



Pre-activation bounds

(can be pre-computed using CROWN)



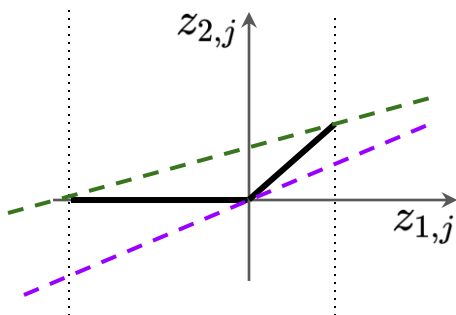
ReLU's lower bound can be optimized (α -CROWN)

Theorem (informal): we can efficiently find D , b' such that:

$$f(x) \geq a^\top W_2 D z_1 + b' \iff f(x) \geq a^\top W_2 z_2 + b \quad \forall x \in \mathcal{S}$$

Illustration: Linear bound propagation process

Proof sketch: conservatively use linear bounds to replace a non-linear function.



$$f(x) \geq a^\top W_2 z_2 + b$$

$$f(x) \geq \sum_j [(a^\top W_2)_j \cdot z_{2,j}] + b$$

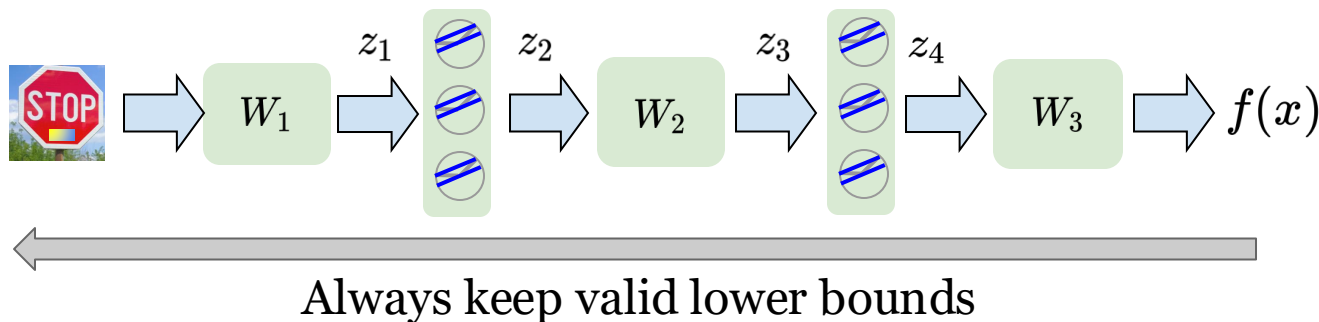
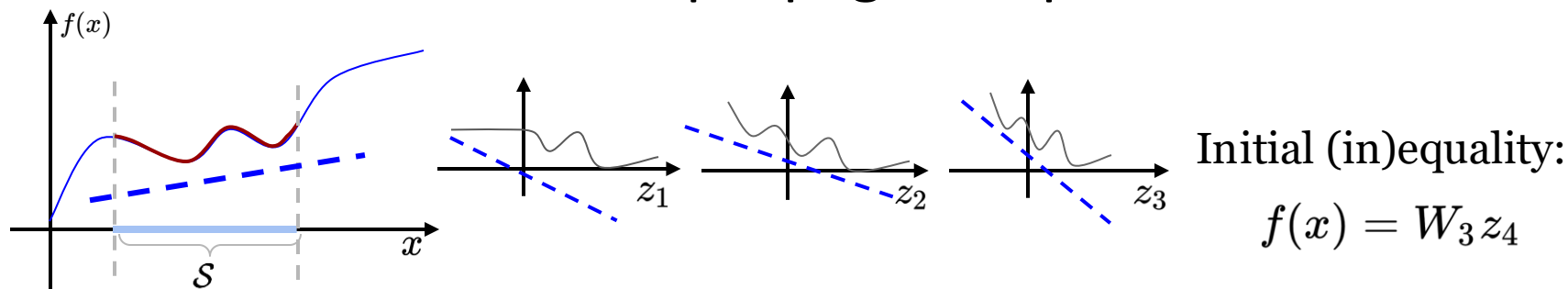
$(a^\top W_2)_j \geq 0$ Choose **lower bound**

$(a^\top W_2)_j < 0$ Choose **upper bound**

Theorem (informal): we can efficiently find D , b' such that:

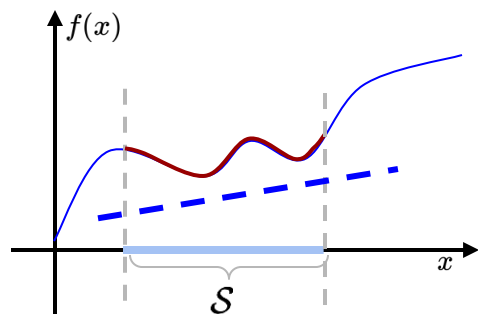
$$f(x) \geq a^\top W_2 D z_1 + b' \longleftarrow f(x) \geq a^\top W_2 z_2 + b \quad \forall x \in \mathcal{S}$$

Illustration: Linear bound propagation process

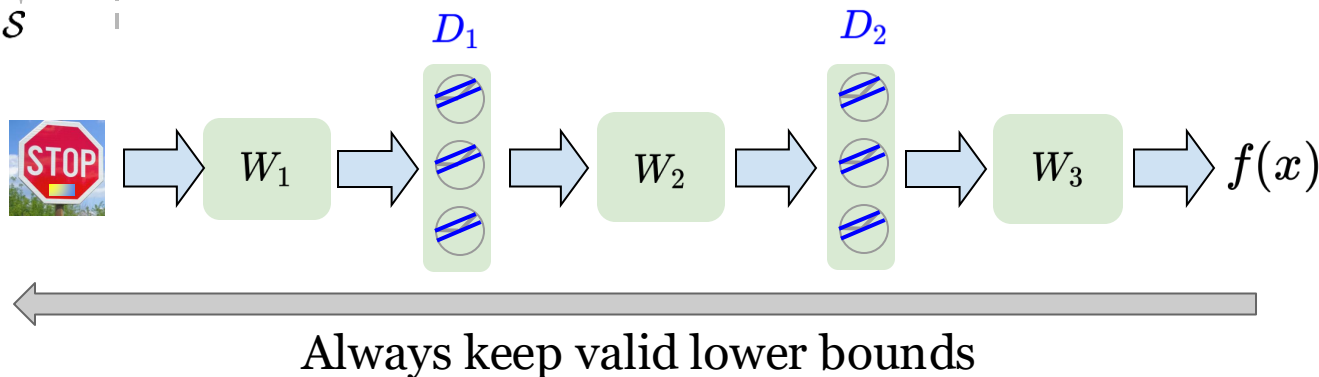


CROWN main theorem (simplified): $f(x) \geq a_{\text{CROWN}}^\top x + b_{\text{CROWN}} \quad \forall x \in \mathcal{S}$

Illustration: Linear bound propagation process



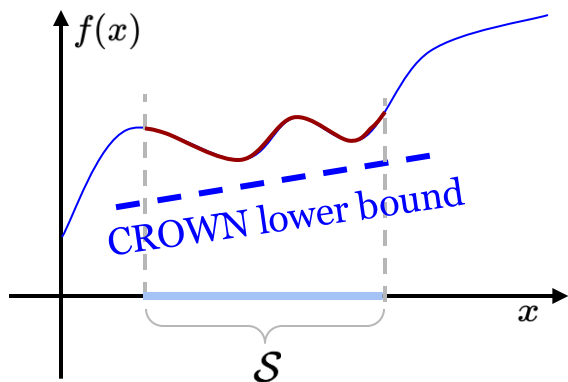
Bounds propagated through
simple matrix multiplications!
Fast and GPU-friendly



CROWN main theorem (simplified): $f(x) \geq a_{\text{CROWN}}^\top x + b_{\text{CROWN}} \quad \forall x \in \mathcal{S}$

$$a_{\text{CROWN}} = W_3 D_2 W_2 D_1 W_1$$

Use Linear Bounds to Prove Robustness



Prove: $\forall x \in \mathcal{S}, f(x) > 0$



$x_1 \in \mathcal{S}$



$x_2 \in \mathcal{S}$



$x_3 \in \mathcal{S}$

...

Lower bound $> 0 \Rightarrow f(x) > 0 \Rightarrow$ verified (always a stop sign)

auto_LiRPA: Verification Library for General Computation Graphs



Colab Demo:

<http://PaperCode.cc/AutoLiRPA-Demo>



The auto_LiRPA library on GitHub:

<http://PaperCode.cc/AutoLiRPA>

MILP/LP vs Bound Propagation

Bound propagation:

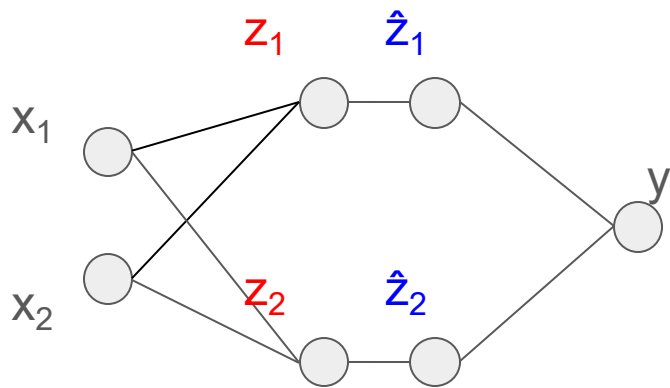
- Scalable and fast propagation
- GPU friendly
- Incomplete verification (will be extended in the next lecture)
- Bounds are looser compared to LP; much looser compared to MILP

MILP/LP:

- Tighter solution
- Does not scale (MILP ~10k neurons, LP ~100k neurons)
- Much slower; cannot utilize GPU

Review: bound propagation with linear bounds (CROWN)

Simple example: linear $\rightarrow$ ReLU $\rightarrow$ linear

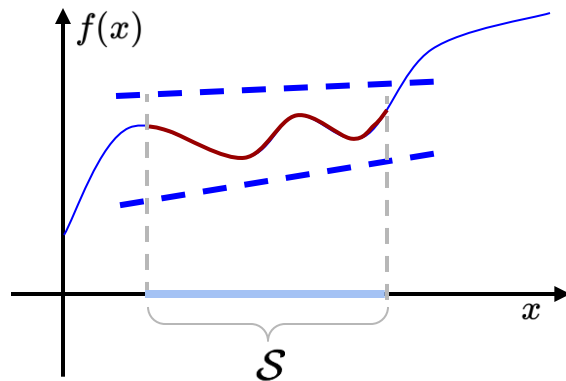


Goal: bound y using symbolic linear functions of x
(linear inequalities)

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

$$z_1 = x_1 - x_2 \quad z_2 = 2x_1 - x_2$$

$$y = \text{ReLU}(z_1) - \text{ReLU}(z_2)$$



Review: bound propagation with linear bounds (CROWN)

Prerequisite: all pre-activation bounds (can be computed using CROWN by treating z_1 and z_2 as the output neuron)

$$x_1 \in [-1, 2], x_2 \in [-2, 1] \quad z_1 = x_1 - x_2 \quad z_2 = 2x_1 - x_2$$

$$z_1 \in [-2, 4] \quad z_2 \in [-3, 6]$$

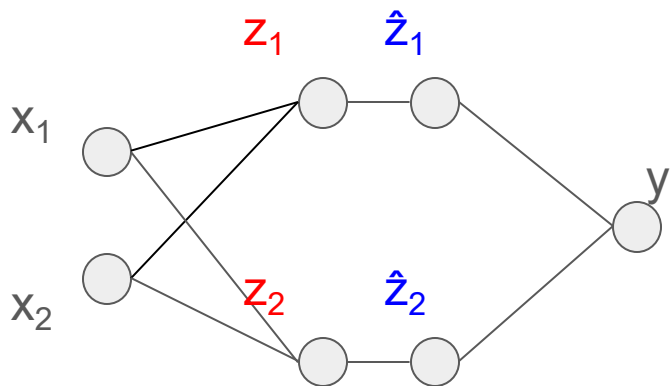
Pre-activation bounds needed for linear bounds
of ReLU or other non-linear functions

Review: bound propagation with linear bounds (CROWN)

Propagation starts from the output y .

Step 1: bound y using linear functions of y (base case): $y \leq y, y \geq y$

Step 2: bound y using linear functions of $\hat{\mathbf{z}}$: simply plugin the definition of the second linear layer:
layer: $y = \hat{z}_1 - \hat{z}_2$



$$y \leq \hat{z}_1 - \hat{z}_2, y \geq \hat{z}_1 - \hat{z}_2$$

Linear layer: simple substitution

Review: bound propagation with linear bounds (CROWN)

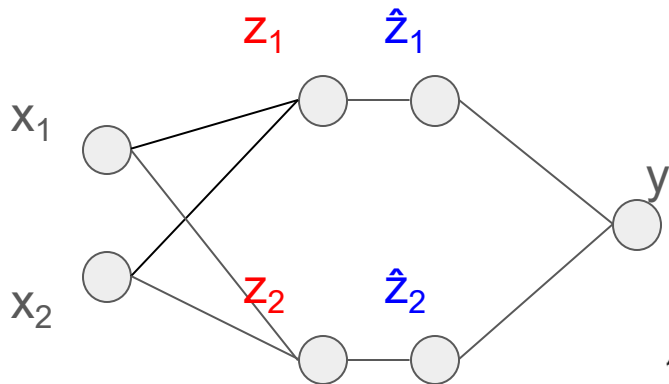
Step 3: bound y using linear functions of z : need linear bounds for ReLU functions, which allows us to replace $\hat{z}$ with z

ReLU layer: use linear bound

Check **sign** of coefficients and take the lower or upper bound

$$y \leq 1 \cdot \hat{z}_1 + (-1) \cdot \hat{z}_2$$

$$y \geq 1 \cdot \hat{z}_1 + (-1) \cdot \hat{z}_2$$

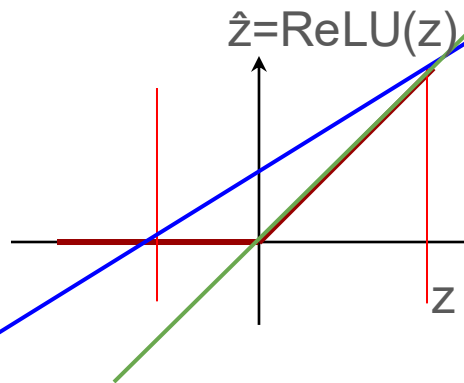


$$z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$



$$z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - z_2$$



Review: bound propagation with linear bounds (CROWN)

Step 4: bound y using linear functions of x

$$z_1 - \left(\frac{2}{3}z_2 + 2\right) \leq y \leq \left(\frac{2}{3}z_1 + \frac{4}{3}\right) - z_2$$

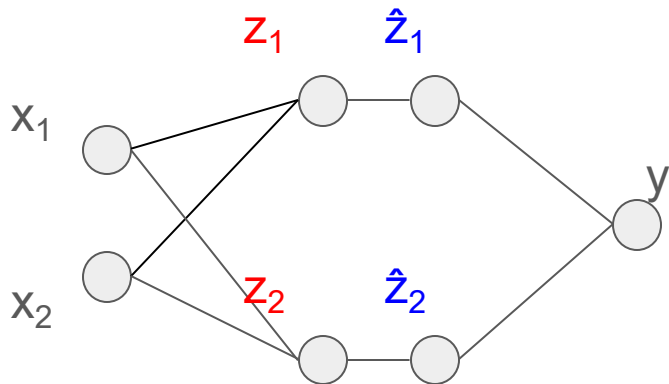
$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$



$$-\frac{1}{3}x_1 - \frac{1}{3}x_2 - 2 \leq y \leq -\frac{4}{3}x_1 + \frac{1}{3}x_2 + \frac{4}{3}$$

Linear layer: simple substitution



Review: bound propagation with linear bounds (CROWN)

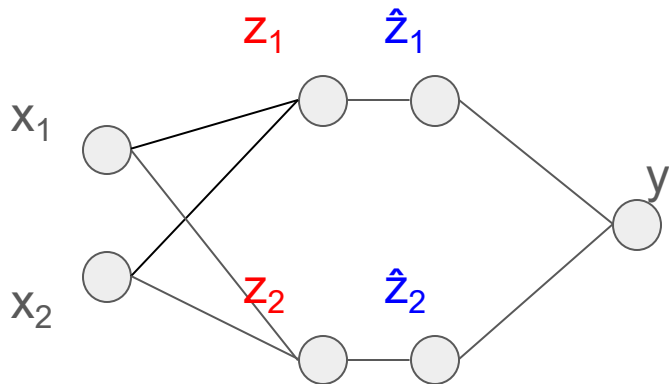
Step 5: concretize linear bounds

$$-\frac{1}{3}x_1 - \frac{1}{3}x_2 - 2 \leq y \leq -\frac{4}{3}x_1 + \frac{1}{3}x_2 + \frac{4}{3}$$

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$



$$y \in [-3, 3]$$



How to improve bound propagation

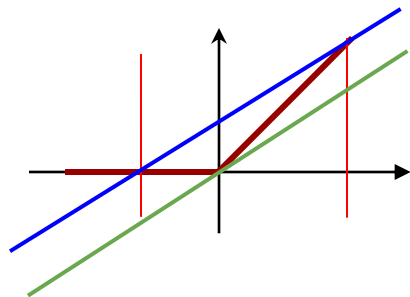
Bound propagation is fast, but what if the bounds are not tight enough?

Goal: use more time to “refine” the bounds. Two techniques:

- Bound optimization (previous lecture)
- **Branch and bound (this lecture)**

Bound optimization (α -CROWN)

In the previous lecture, we discussed the possibility of making the lower bound of a ReLU function optimizable. α can be optimization used gradient descent.

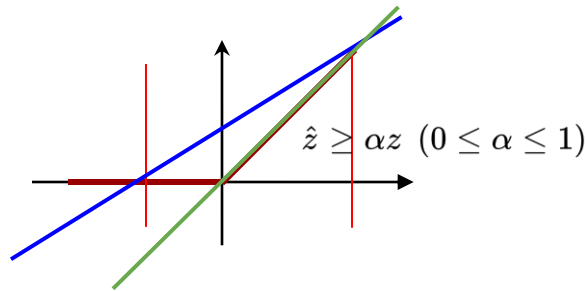


$$\frac{2}{3}z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$\frac{2}{3}z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$

↓

$$y \in \left[-\frac{10}{3}, 2\right]$$



$$z_1 \leq \text{ReLU}(z_1) \leq \frac{2}{3}z_1 + \frac{4}{3}$$

$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$

↓

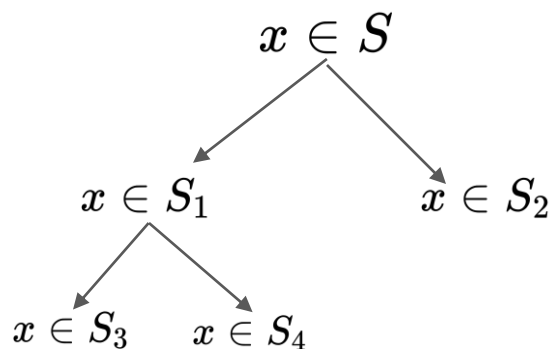
$$y \in [-3, 3]$$

Branch and bound

General idea: split (branch) the original problem into easier subproblems; obtain bounds on each subproblem

Define **LB(S)** as the lower bound obtained using bound propagation for

$$\min_{x \in S} f(x)$$



$$S_1 \cup S_2 = S$$

$$LB(S)$$

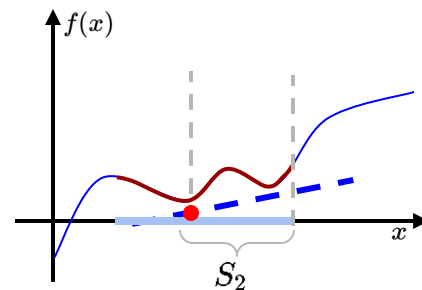
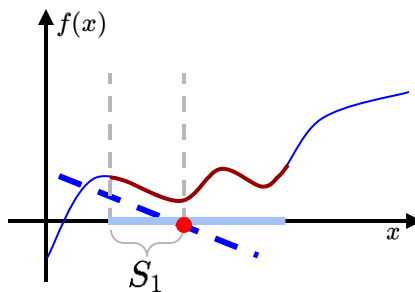
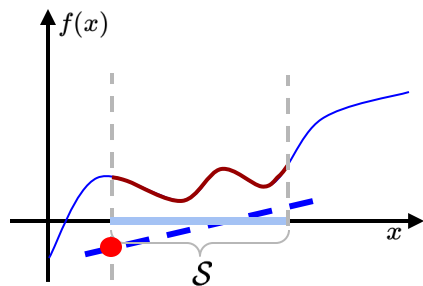
$$\min(LB(S_1), LB(S_2))$$

$$S_3 \cup S_4 \cup S_2 = S$$

$$\min(LB(S_3), LB(S_4), LB(S_2))$$

All leaf nodes

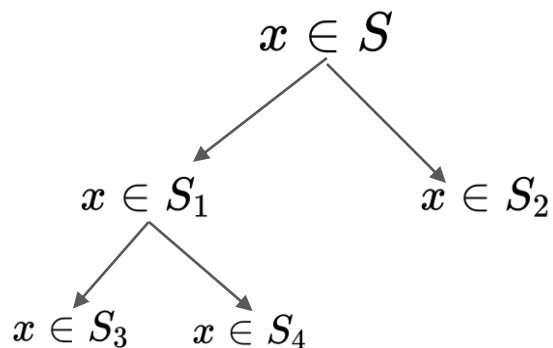
Branch and bound: why the lower bounds become tighter?



$LB(S)$

$\min(LB(S_1), LB(S_2))$

$\min(LB(S_3), LB(S_4), LB(S_2))$

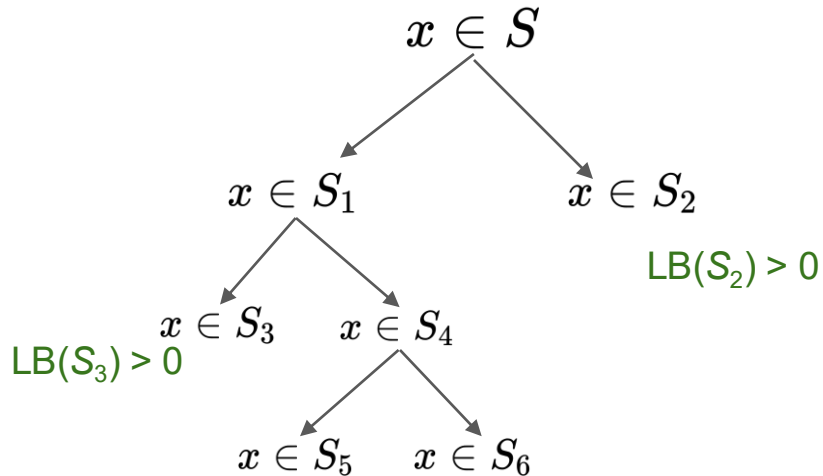


$S_1 \cup S_2 = S$

$S_3 \cup S_4 \cup S_2 = S$

Branch and bound

If $LB(S_i) > 0$, it can be removed from our problem since the property is verified on this subdomain S_i ; branch and bound is needed for unverified subdomains only.



List of unverified subproblems

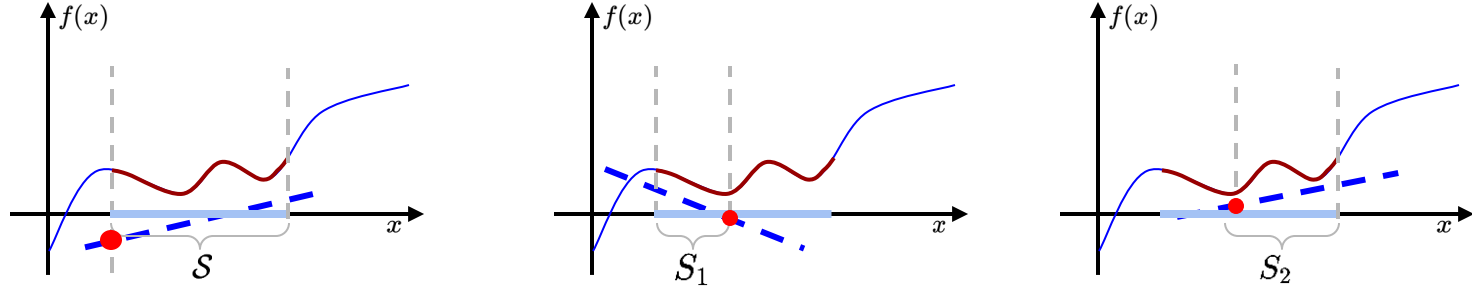
$\{S\}$

~~$\{S_1, S_2\}$~~

~~$\{S_3, S_4\}$~~

$\{S_5, S_6\}$

Branch and bound on input



Split each into domain S , typically by

$$S = \{x_1 \in [-1, 1], x_2 \in [-1, 1]\} \Rightarrow$$

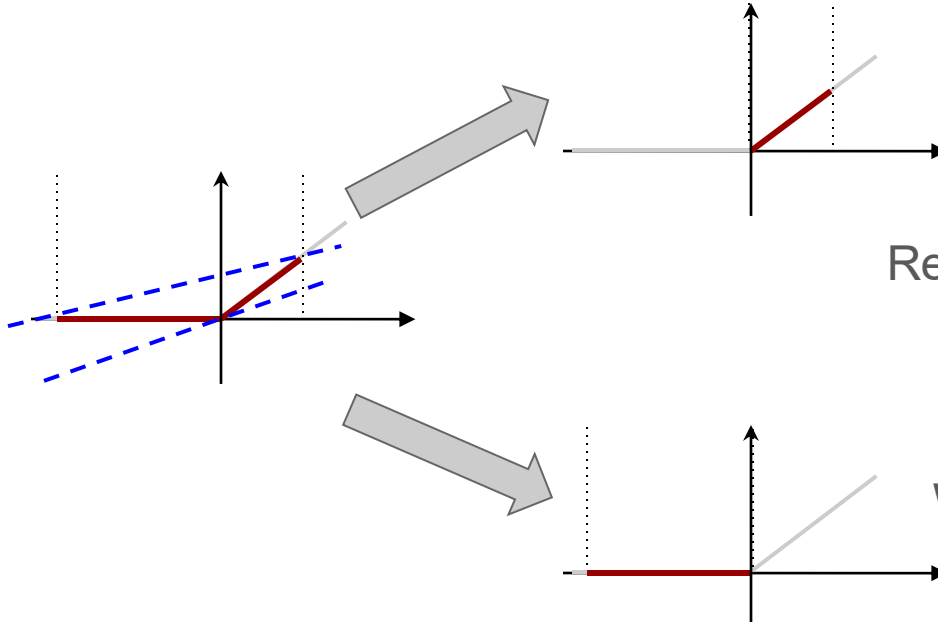
$$S_1 = \{x_1 \in [-1, 0], x_2 \in [-1, 1]\}, S_2 = \{x_1 \in [0, 1], x_2 \in [-1, 1]\}$$

Implementation is easy

Does not work well when input dimension is very high (e.g., image inputs)

Branch and bound on ReLU

Implicitly split input domain S by considering a ReLU neuron in two cases: active and inactive.



ReLU becomes **linear** in both subproblems

Works best when the number of unstable neurons is not very large

Branch and bound on ReLU

Implicitly split input domain S by considering a ReLU neuron in two cases: active and inactive.

$$S = \{x_1 \in [-1, 1], x_2 \in [-1, 1]\} \Rightarrow$$

$$S_1 = \{x_1 \in [-1, 1], x_2 \in [-1, 1], z^{(i)}_j(x_1, x_2) \geq 0\},$$

z is a function of input x

$$S_2 = \{x_1 \in [-1, 1], x_2 \in [-1, 1], z^{(i)}_j(x_1, x_2) \leq 0\}$$

E.g., for our example $x_1 \in [-1, 2], x_2 \in [-2, 1] \quad z_1 = x_1 - x_2$

Splitting z_1 essentially consider two cases $x_1 - x_2 \geq 0$ and $x_1 - x_2 \leq 0$

Let's go over our example again with split

Prerequisite: all pre-activation bounds

$$x_1 \in [-1, 2], x_2 \in [-2, 1] \quad z_1 = x_1 - x_2 \quad z_2 = 2x_1 - x_2$$

Split constraint: $z_1 \leq 0$

$$z_1 \in [-2, 0] \quad z_2 \in [-3, 6]$$

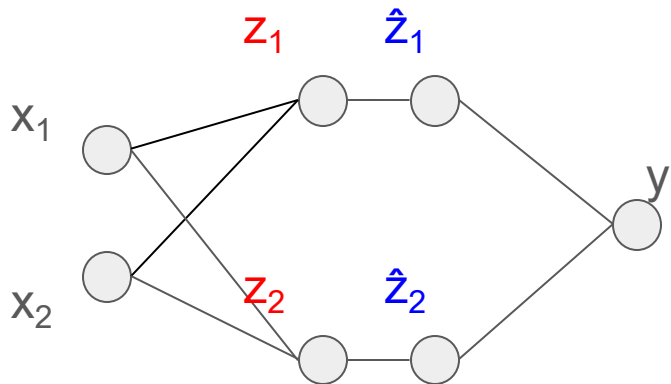
Pre-activation bounds for z_1 updated!

CROWN with neuron split

Let's look at the **lower bound** only (since only lower bound is needed)

Step 1: bound y using linear functions of y (base case): $y \leq y$, $y \geq y$

Step 2: bound y using linear functions of $\hat{\mathbf{z}}$: simply plugin the definition of the second linear layer:
layer: $y = \hat{z}_1 - \hat{z}_2$



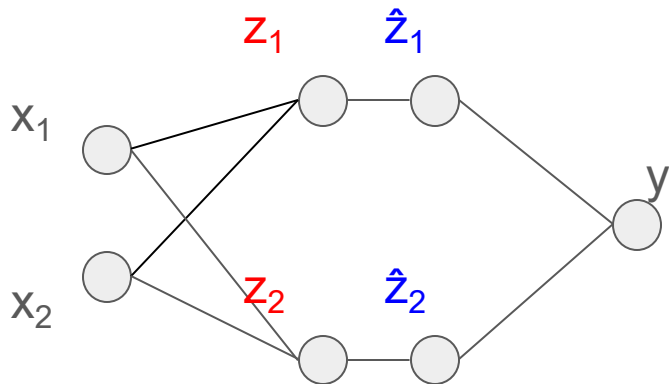
$$y \geq \hat{z}_1 - \hat{z}_2$$

Linear layer: simple substitution

CROWN with neuron split (changed with the split)

Step 3: bound y using linear functions of z : need linear bounds for ReLU functions, which allows us to replace $\hat{z}$ with z

ReLU layer: use linear bound
Check **sign** of coefficients and take
the lower or upper bound



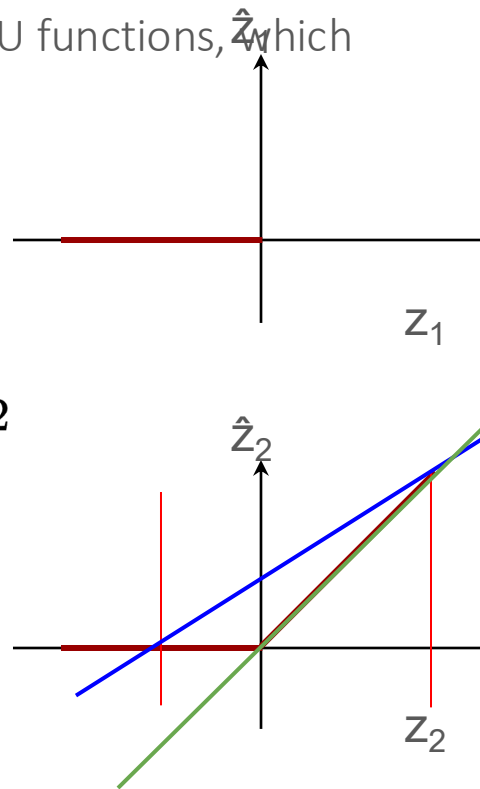
$$y \geq 1 \cdot \hat{z}_1 + (-1) \cdot \hat{z}_2$$

$$0 \leq \text{ReLU}(z_1) \leq 0$$

$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$

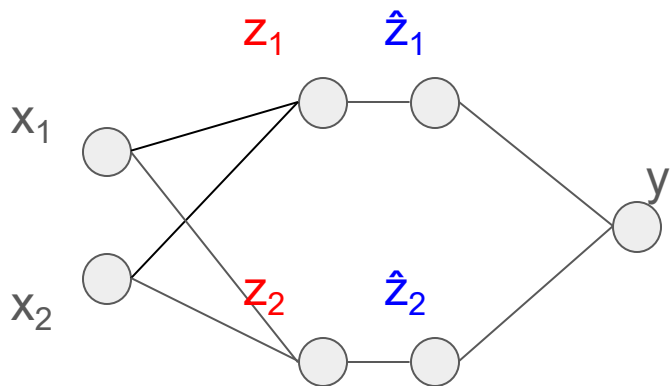


$$y \geq -\frac{2}{3}z_2 - 2$$



CROWN with neuron split (changed with the split)

Step 4: bound y using linear functions of $\mathbf{x}$



$$y \geq -\frac{2}{3}z_2 - 2$$

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$

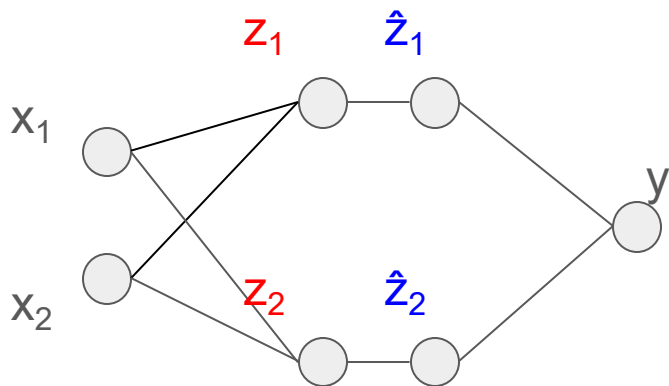


$$y \geq -\frac{4}{3}x_1 + \frac{2}{3}x_2 - 2$$

Linear layer: simple substitution

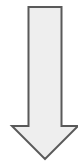
CROWN with neuron split (changed with the split)

Step 5: concretize linear bounds



$$y \geq -\frac{4}{3}x_1 + \frac{2}{3}x_2 - 2$$

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$



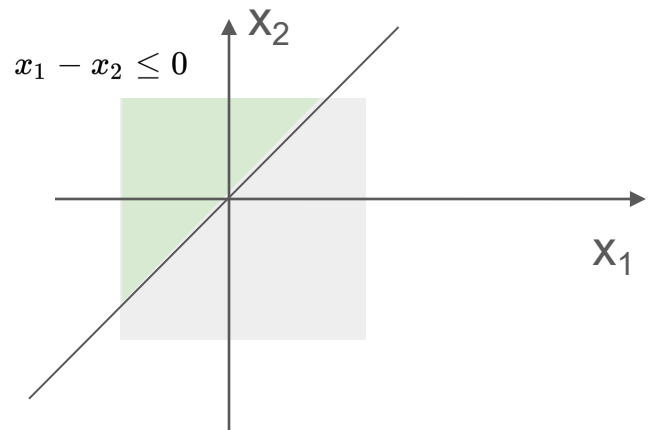
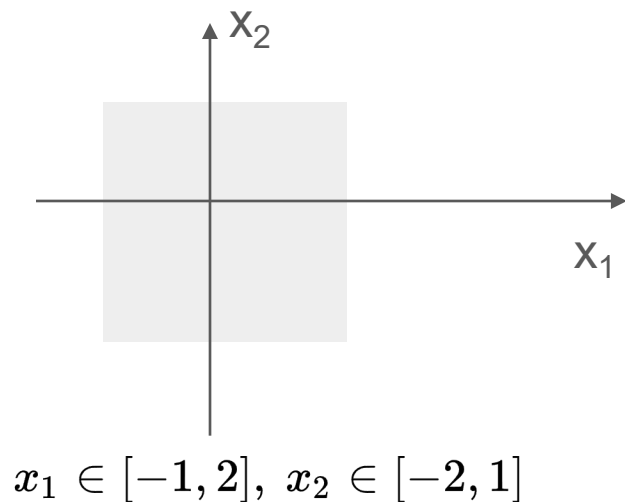
$$y \geq -6$$

Recall that without the split, we have $y \geq -3$
With the split we expect the lower bound to improve??

What is going wrong with CROWN?

The split constraint is not fully used during the process.

$$z_1 \leq 0 \implies x_1 - x_2 \leq 0$$

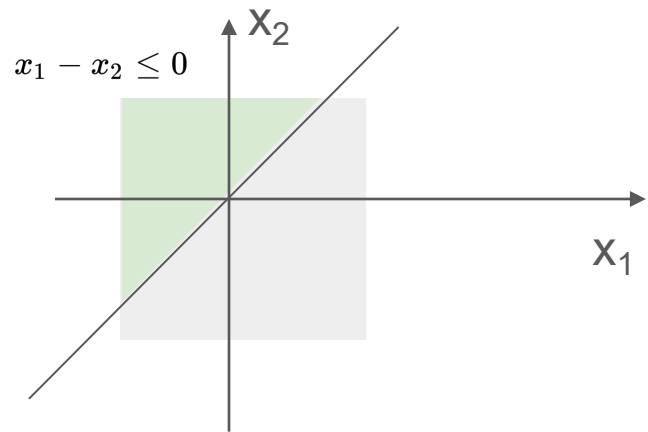


What is going wrong with CROWN?

In the concretization step, we still consider the worst case scenario in the larger box, rather than the green triangle.

$$y \geq -\frac{4}{3}x_1 + \frac{2}{3}x_2 - 2$$

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$



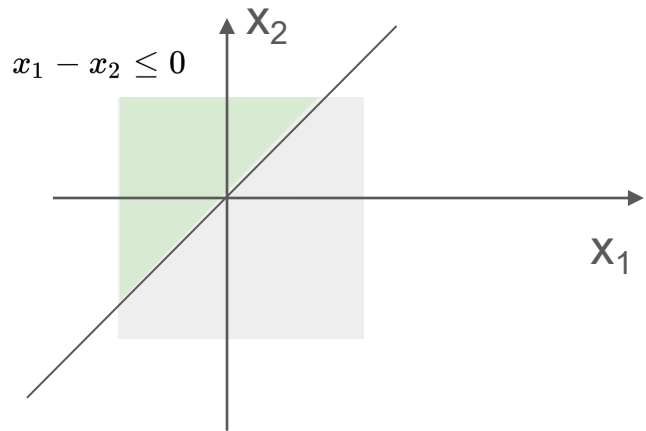
How to address the problem?

Instead we should solve this optimization problem during concretization:

$$\min_{x_1, x_2} -\frac{4}{3}x_1 + \frac{2}{3}x_2 - 2$$

$$\text{s.t. } z_1 \leq 0$$

$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$



CROWN cannot handle this **constraint!**

β -CROWN: bound propagation with split constraint

We use *Lagrangian multipliers* to handle this constraint.

To solve a constrained optimization problem:

$$\begin{aligned} & \min f_0(x) \\ & \text{such that } f_i(x) \leq 0 \qquad \forall i \in 1, \dots, m \end{aligned}$$

We can define Lagrangian with $\lambda_i \geq 0$:

$$L(x, \lambda) = f_0(x) + \sum_i \lambda_i f_i(x)$$

So the optimization problem can be written as

$$\min_x \max_{\lambda} L(x, \lambda)$$

β -CROWN: bound propagation with split constraint

$$\min_x \max_{\lambda} L(x, \lambda)$$

It is hard to solve directly. But we can then apply *weak duality*, which gives a lower bound

$$\max_{\lambda} \min_x L(x, \lambda) \leq \min_x \max_{\lambda} L(x, \lambda)$$

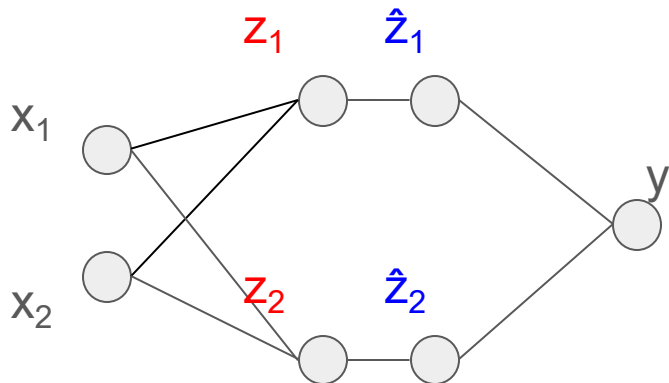
It has an intuitive game-theoretic explanation: whoever plays second may have an advantage, because they know the move of the first player.

Closed form solution exist for the inner minimization
(basically the concretization process without constraints)

β -CROWN with neuron split

Step 3: bound y using linear functions of z : need linear bounds for ReLU functions, which allows us to replace $\hat{z}$ with z

ReLU layer: use linear bound
Check **sign** of coefficients and take
the lower or upper bound



$$y \geq 1 \cdot \hat{z}_1 + (-1) \cdot \hat{z}_2$$

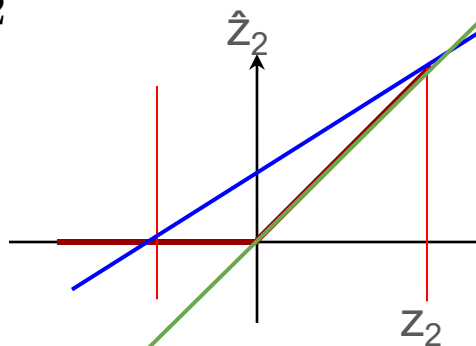
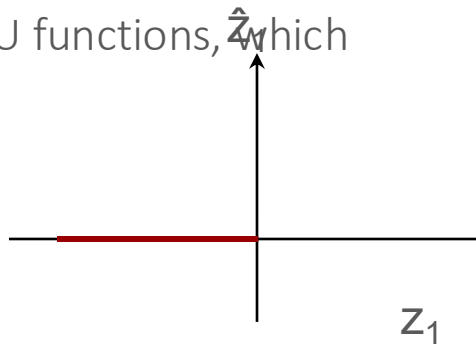
$$0 \leq \text{ReLU}(z_1) \leq 0$$

$$z_2 \leq \text{ReLU}(z_2) \leq \frac{2}{3}z_2 + 2$$



$$y \geq -\frac{2}{3}z_2 - 2 + \beta z_1$$

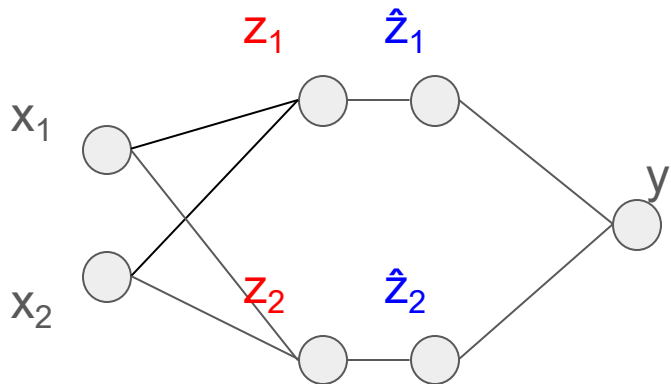
$\beta \geq 0$



Change in bound propagation: add β for each split constraint

β -CROWN with neuron split

Step 4: bound y using linear functions of $\mathbf{x}$, Now our bound has a parameter β



$$y \geq -\frac{2}{3}z_2 - 2 + \beta z_1$$

$$z_1 = x_1 - x_2$$

$$z_2 = 2x_1 - x_2$$



$$y \geq (\beta - \frac{4}{3})x_1 + (\frac{2}{3} - \beta)x_2 - 2$$

Linear layer: simple substitution

β -CROWN with neuron split

Step 5: concretize linear bounds

$$y \geq (\beta - \frac{4}{3})x_1 + (\frac{2}{3} - \beta)x_2 - 2$$
$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

Concretization depends on the sign of the coefficients, so we must discuss three cases:

$$0 \leq \beta \leq \frac{2}{3}$$
$$\frac{2}{3} \leq \beta \leq \frac{4}{3}$$
$$\beta \geq \frac{4}{3}$$

β -CROWN with neuron split

Step 5: concretize linear bounds

$$y \geq (\beta - \frac{4}{3})x_1 + (\frac{2}{3} - \beta)x_2 - 2$$
$$x_1 \in [-1, 2], x_2 \in [-2, 1]$$

$$0 \leq \beta \leq \frac{2}{3}$$

$$y \geq (\beta - \frac{4}{3}) \cdot 2 + (\frac{2}{3} - \beta) \cdot (-2) - 2$$

The optimal β to maximize y is $2/3$, with objective = $-10/3$

$$\frac{2}{3} \leq \beta \leq \frac{4}{3}$$

$$y \geq (\beta - \frac{4}{3}) \cdot 2 + (\frac{2}{3} - \beta) \cdot 1 - 2$$

The optimal β is $4/3$, with objective = **$-8/3$**

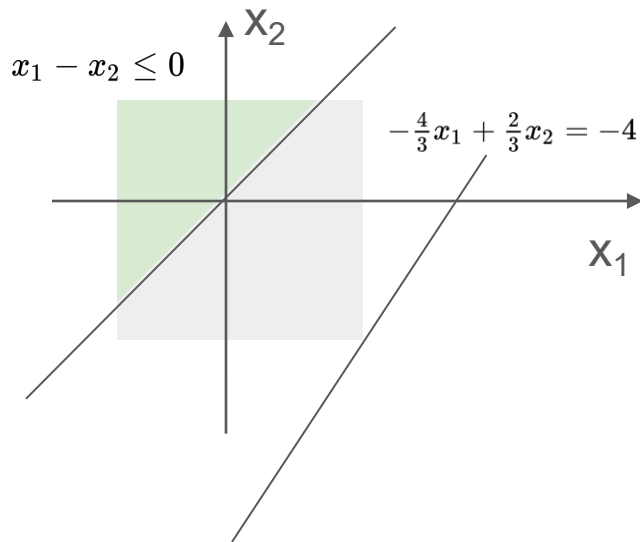
$$\beta \geq \frac{4}{3}$$

$$y \geq (\beta - \frac{4}{3}) \cdot (-1) + (\frac{2}{3} - \beta) \cdot 1 - 2$$

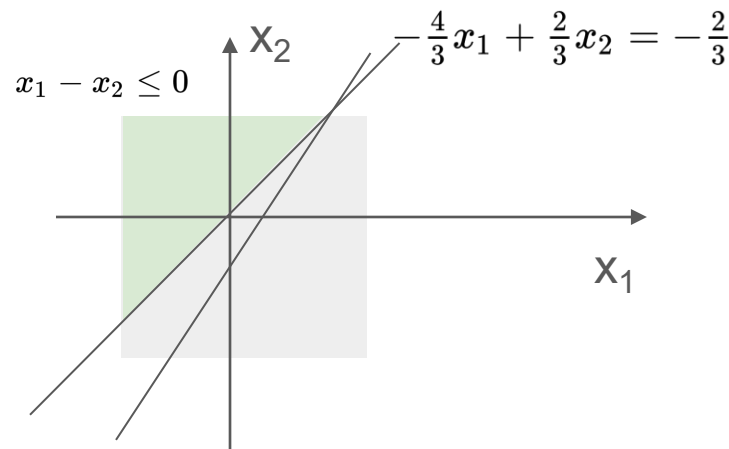
The optimal β is $4/3$, with objective = **$-8/3$**

Geometric interpretation

$$\min_{x_1, x_2} -\frac{4}{3}x_1 + \frac{2}{3}x_2 - 2$$



No constraint, obj = -6



With constraint, obj = -8/3, improved!

Which dimension/which neuron to branch?

Similar to the backtracking process in DPLL, the selection of which dimension (for input split) or which neuron (ReLU split) is very important.

Strong branching: try every possible branch and choose the one with actual largest improvements in lower bound

Heuristic branching: estimate how good a branch is, and choose the neuron/dimension with highest score.

Example branching heuristic

$$S = \{x_1 \in [-1, 1], x_2 \in [-1, 1]\} \Rightarrow$$

$$S_1 = \{x_1 \in [-1, 0], x_2 \in [-1, 1]\}, S_2 = \{x_2 \in [0, 1], x_1 \in [-1, 1]\}$$

OR

$$S_1 = \{x_1 \in [-1, 1], x_2 \in [-1, 0]\}, S_2 = \{x_2 \in [-1, 1], x_1 \in [0, 1]\}$$

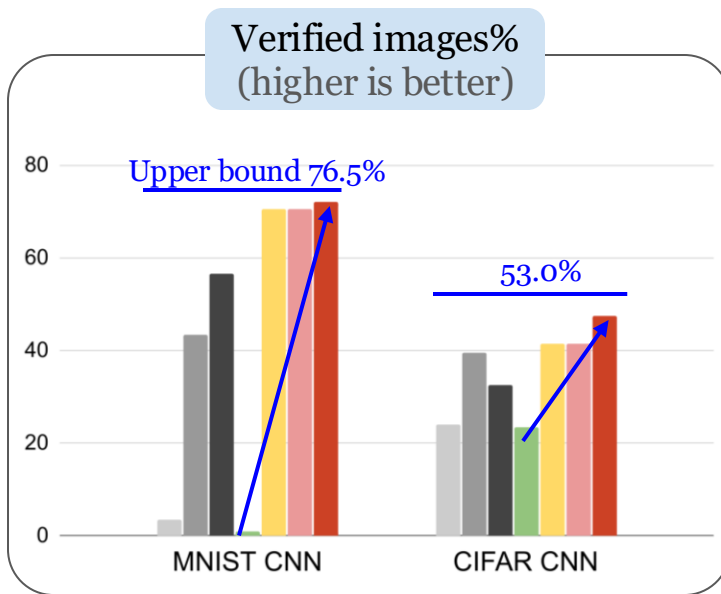
We can estimate the impact on lower bound given changes on x_1 and x_2

Given the CROWN linear bound $y \geq a_1 x_1 + a_2 x_2 + c$, we branch on dimension i where $|a_i|$ is largest.

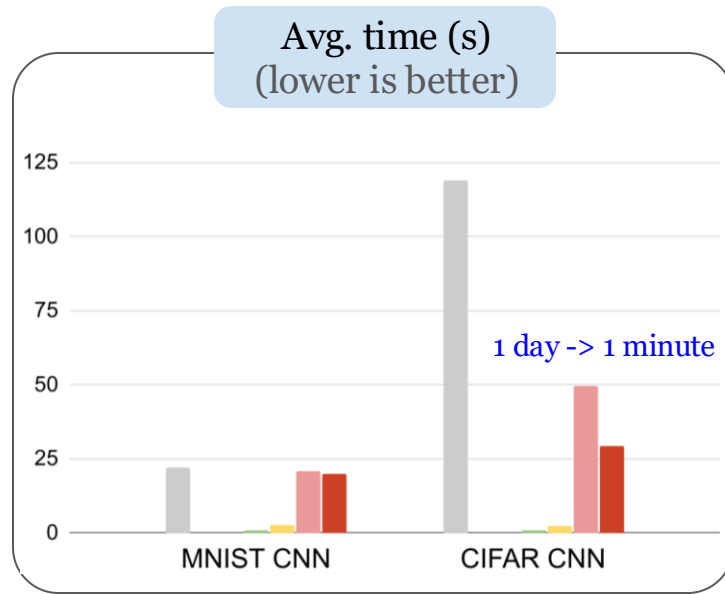
Benchmarks: CROWN-family bound propagation algorithms

- Linear Programming (Salman et al. 2019)
- Semidefinite Programming (Dathathri et al. 2020)
- Integer Programming (Tjeng et al. 2017)
- CROWN
- α -CROWN
- β -CROWN
- GCP-CROWN

Pixel perturbation magnitude:
0.3 for MNIST, 2/255 for CIFAR



Model size: ~5k neurons



Integer programming and semidefinite programming **not plotted** (~1 day)

Key enablers: specialized bound propagation solver + GPU acceleration + BaB

Theoretical Connections: CROWN vs MIP/LP

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

